



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №7
по дисциплине «Технологии и инструментарий машинного обучения»

Студент группы *ИНБО-20-23. Боргачев Т.М.*

(подпись)

Преподаватель *Трушин СМ*

(подпись)

Москва 2025 г.

СОДЕРЖАНИЕ

1 ПРАКТИЧЕСКАЯ РАБОТА №7	3
1.1 Задание 1. Отбор признаков по корреляции и простым критериям	3
1.1.1 Выбор датасета	3
1.1.2 Предобработка данных и EDA.....	5
1.1.3 Отбор признаков	7
1.1.4 Проверка	14
1.2 Задание 2. Сравнение методов отбора на ансамблевых моделях	18
1.2.1 Построение базовых моделей	18
1.2.2 Отбор признаков	19
1.3 Задание 3. Интерпретируемый отбор признаков	32
1.3.1 Предобработка данных и EDA со сложным и «грязным» датасетом	32
1.3.2 Обучение моделей	37
1.3.3 Интерпретация и отбор	41
1.3.4 Сравнение признаков	49
ВЫВОД.....	54

1 ПРАКТИЧЕСКАЯ РАБОТА №7

1.1 Задание 1. Отбор признаков по корреляции и простым критериям

1.1.1 Выбор датасета

Ссылка на Google colab с выполненной работой:
<https://colab.research.google.com/drive/1pzvOexQXKPWFiG0EUcTfdL7cnU8VZ9qS?usp=sharing>.

В качестве исходных данных был использован датасет «Human Activity Recognition». Эксперименты были проведены с группой из 30 добровольцев в возрасте от 19 до 48 лет. Каждый участник выполнял шесть действий (ХОДИЛ, ПОДНИМАЛСЯ ПО ЛЕСТНИЦЕ, СПУСКАЛСЯ ПО ЛЕСТНИЦЕ, СИДЕЛ, СТОЯЛ, ЛЕЖАЛ), используя смартфон на поясе.

Используя встроенный акселерометр и гироскоп, мы зафиксировали 3-осевое линейное ускорение и 3-осевую угловую скорость с постоянной частотой 50 Гц. Эксперименты были записаны на видео, чтобы вручную пометить данные. Полученный набор данных был случайным образом разделен на два набора, из которых 70% добровольцев были отобраны для генерации обучающих данных, а 30% - для тестирования.

Код для обзора данных представлен в Листинге 1.

Листинг 1 — Загрузка данных

```
# Импорт библиотек для работы с данными и визуализации
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_openml
```

Продолжение Листинга 1

```
# Загрузка датасета Human Activity Recognition
# Целевая переменная: 6 типов активности (WALKING,
WALKING_UPSTAIRS, WALKING_DOWNSTAIRS, SITTING, STANDING, LAYING)
human_activity = fetch_openml('har', version=1)
X = pd.DataFrame(human_activity.data,
columns=human_activity.feature_names)
y = pd.Series(human_activity.target)

df = pd.concat([X, y.rename('activity')], axis=1)

print(f"Размер данных: {df.shape}")
print("\nПервые 5 строк данных:")
print(df.head())

print("\nТипы данных:")
print(df.dtypes.value_counts())

print("\nКоличество уникальных значений в целевой переменной:")
print(df['activity'].value_counts())
```

Результат представлен на Рисунке 1.

```
Размер данных: (10299, 562)

Первые 5 строк данных:
   V1      V2      V3      V4      V5      V6      V7  \
0  0.288585 -0.020294 -0.132905 -0.995279 -0.983111 -0.913526 -0.995112
1  0.278419 -0.016411 -0.123520 -0.998245 -0.975300 -0.960322 -0.998807
2  0.279653 -0.019467 -0.113462 -0.995380 -0.967187 -0.978944 -0.996520
3  0.279174 -0.026201 -0.123283 -0.996091 -0.983403 -0.990675 -0.997099
4  0.276629 -0.016570 -0.115362 -0.998139 -0.980817 -0.990482 -0.998321

   V8      V9      V10  ...  V553  V554  V555  V556  \
0 -0.983185 -0.923527 -0.934724  ... -0.298676 -0.710304 -0.112754  0.030400
1 -0.974914 -0.957686 -0.943068  ... -0.595051 -0.861499  0.053477 -0.007435
2 -0.963668 -0.977469 -0.938692  ... -0.390748 -0.760104 -0.118559  0.177899
3 -0.982750 -0.989303 -0.938692  ... -0.117290 -0.482845 -0.036788 -0.012892
4 -0.979672 -0.990441 -0.942469  ... -0.351471 -0.699205  0.123320  0.122542

   V557  V558  V559  V560  V561  activity
0 -0.464761 -0.018446 -0.841247  0.179941 -0.058627      5
1 -0.732626  0.703511 -0.844788  0.180289 -0.054317      5
2  0.100699  0.808529 -0.848933  0.180637 -0.049118      5
3  0.640011 -0.485366 -0.848649  0.181935 -0.047663      5
4  0.693578 -0.615971 -0.847865  0.185151 -0.043892      5

[5 rows x 562 columns]

Типы данных:
float64      561
category      1
Name: count, dtype: int64
```

Рисунок 1 — Обзор данных

Перейдем к очистке данных.

1.1.2 Предобработка данных и EDA

Код для предобработки данных представлен в Листинге 2.

Листинг 2 — Предобработка данных

```
# Импорт библиотек для визуализации и предобработки
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.model_selection import train_test_split

# Отображение базовых статистик
print("Базовые статистики числовых признаков:")
print(df.describe())

# Проверка на пропущенные значения
missing_values = df.isnull().sum().sum()
print(f"\nКоличество пропущенных значений в датасете: {missing_values}")

# Визуализация распределения целевой переменной
plt.figure(figsize=(10, 6))
sns.countplot(x='activity', data=df)
plt.title('Распределение классов активности')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Визуализация корреляции признаков
# Возьмем первые 15 признаков для корреляционной матрицы
plt.figure(figsize=(16, 12))
subset_features = df.columns[:15].tolist() + ['activity']
correlation_matrix = df[subset_features].corr(numeric_only=True)
sns.heatmap(correlation_matrix, annot=True, fmt=".2f",
            cmap='coolwarm',
            annot_kws={'size': 8}, cbar_kws={'label': 'Коэффициент корреляции'})
plt.title('Корреляционная матрица (подмножество признаков)')
plt.tight_layout()
plt.show()

# Преобразование целевой переменной в числовой формат
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['activity_encoded'] = le.fit_transform(df['activity'])
y_encoded = df['activity_encoded']
```

Продолжение Листинга 2

```
# Разделение данных на обучающую и тестовую выборки
X = df.drop(['activity', 'activity_encoded'], axis=1)
X_train, X_test, y_train, y_test = train_test_split(X, y_encoded,
test_size=0.2, random_state=42)

# Масштабирование данных
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nДанные успешно предобработаны и разделены на обучающую и
тестовую выборки.")
print(f"Размер обучающей выборки: {X_train_scaled.shape}")
print(f"Размер тестовой выборки: {X_test_scaled.shape}")
```

В результате количество пропущенных значений в датасете: 0.

Распределение классов представлено на Рисунке 2.

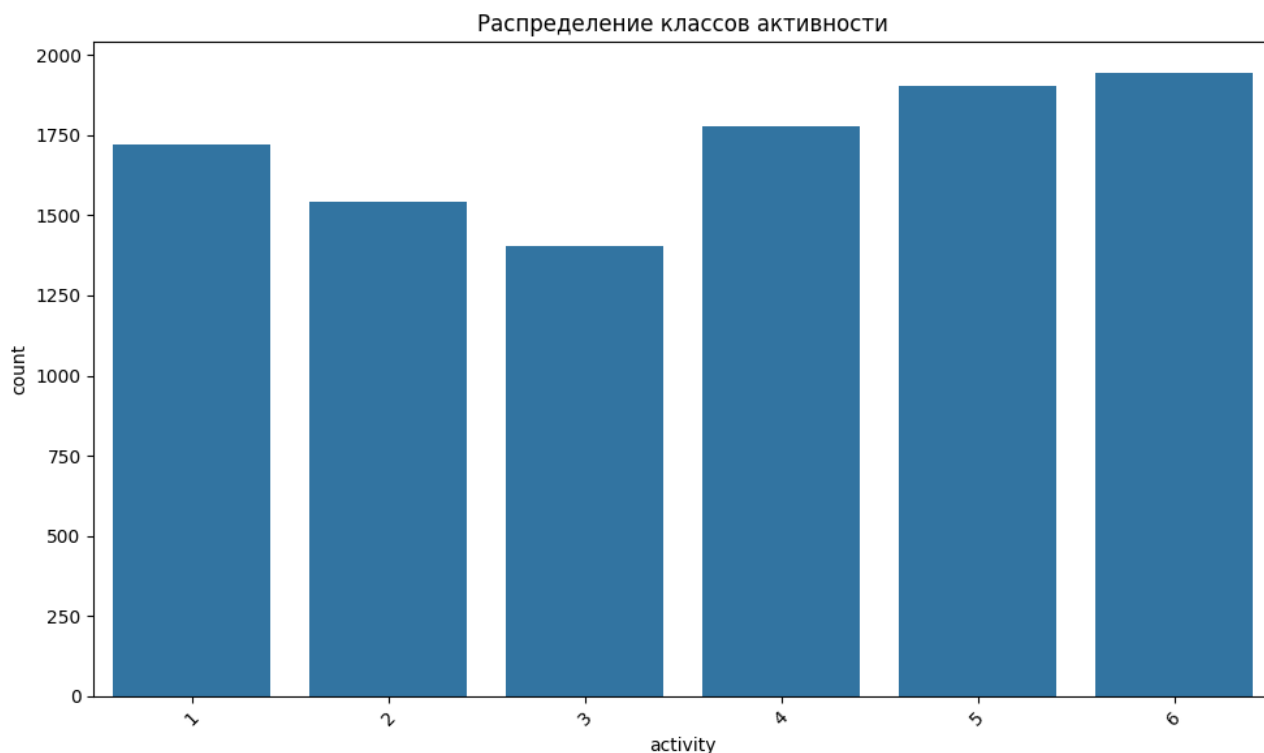


Рисунок 2 — Распределение классов

Матрица корреляции для признаков представлена на Рисунке 3.

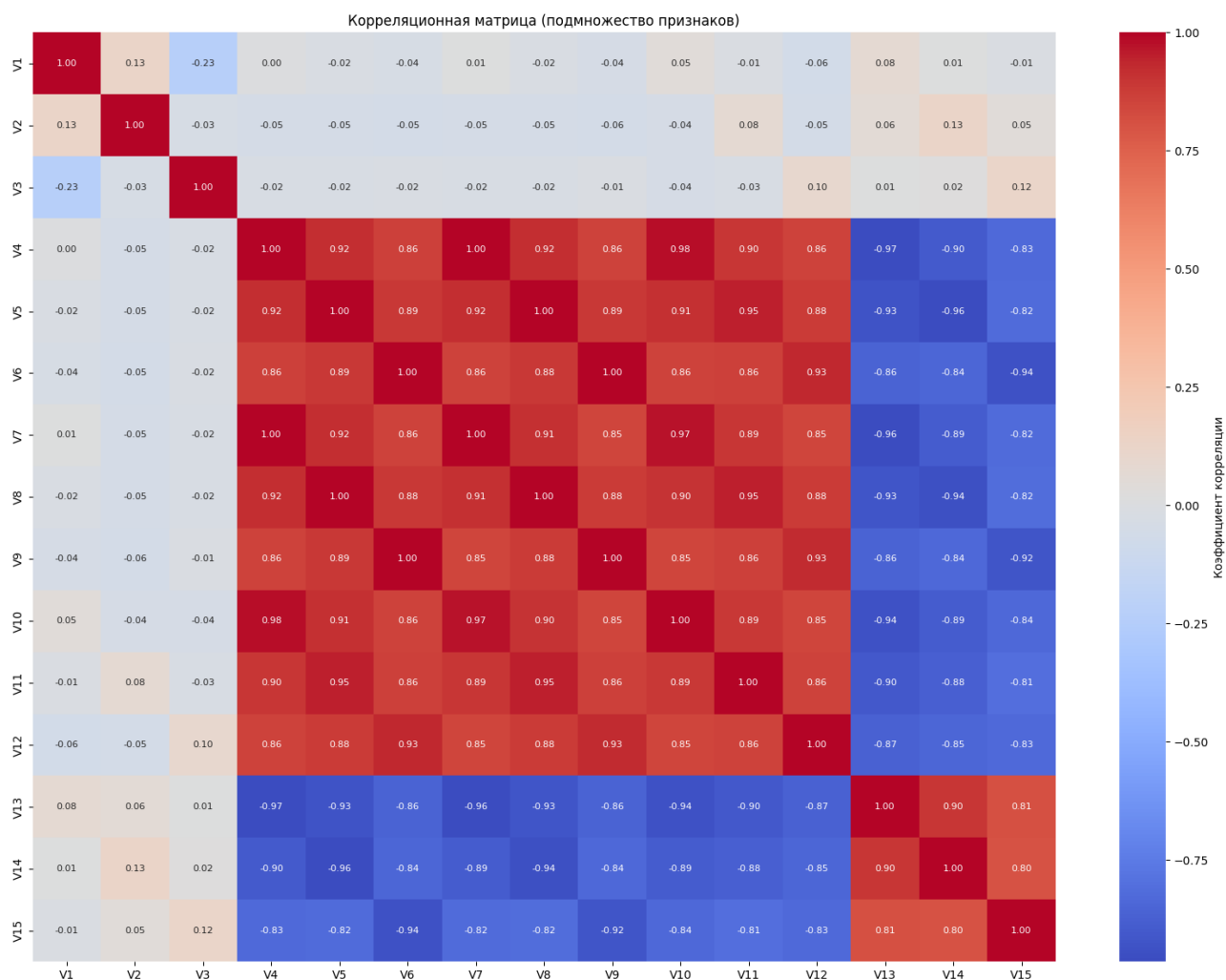


Рисунок 3 — Матрица корреляции

Среди данных явно видна высокая мультиколлинеарность.

Данные успешно предобработаны и разделены на обучающую и тестовую выборки.

Размер обучающей выборки: (8239, 561).

Размер тестовой выборки: (2060, 561).

1.1.3 Отбор признаков

Код для отбора признаков с помощью анализа корреляции представлен в Листинге 3.

Листинг 3 — Отбор признаков с помощью корреляционной матрицы

```
# Импорт библиотек для отбора признаков
from sklearn.feature_selection import SelectKBest, f_classif,
mutual_info_classif, chi2

X_train_df = pd.DataFrame(X_train_scaled, columns=X.columns)

# Добавление целевой переменной для корреляционного анализа
X_train_df['target'] = y_train.values

correlations = X_train_df.corr()['target'].sort_values(ascending=False)
correlations = correlations.drop('target')

# Визуализация корреляций
plt.figure(figsize=(15, 8))
correlations.abs().sort_values(ascending=False).head(20).plot(kind='bar')
plt.title('Абсолютные значения корреляций 20 наиболее значимых признаков с целевой переменной')
plt.ylabel('Абсолютное значение корреляции')
plt.tight_layout()
plt.show()

# Анализ корреляций между признаками для выявления мультиколлинеарности
# Выберем признаки с корреляцией > 0.85
corr_matrix = X_train_df.drop('target', axis=1).corr().abs()
upper_tri = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))

# Найдем пары признаков с высокой корреляцией
high_corr_pairs = []
for row in range(len(upper_tri.columns)):
    for col in range(row+1, len(upper_tri.columns)):
        if upper_tri.iloc[row, col] > 0.85:
            high_corr_pairs.append((upper_tri.columns[row], upper_tri.columns[col], upper_tri.iloc[row, col]))

print(f"\nНайдено {len(high_corr_pairs)} пар признаков с корреляцией > 0.85:")
for pair in high_corr_pairs[:10]:
    print(f"{pair[0]} и {pair[1]}: корреляция = {pair[2]:.3f}")

# Удалим один из признаков из каждой высокоррелирующей пары
high_corr_features = set()
for pair in high_corr_pairs:
    # Оставим признак с большей корреляцией с целевой переменной
    corr1 = abs(correlations[pair[0]])
    corr2 = abs(correlations[pair[1]])
```


Продолжение Листинга 3

```
if corr1 < corr2:
    high_corr_features.add(pair[0])
else:
    high_corr_features.add(pair[1])

print(f"\nКоличество признаков для удаления из-за высокой
корреляции: {len(high_corr_features)}")
print("Признаки для удаления:", list(high_corr_features)[:10])

# Применение порога корреляции с целевой переменной
# Оставим признаки с корреляцией > 0.1 по абсолютному значению
threshold = 0.1
significant_features = correlations[abs(correlations) >
threshold].index
print(f"\nКоличество признаков с корреляцией > {threshold} с
целевой переменной: {len(significant_features)}")

# Финальный набор признаков после корреляционного отбора
final_corr_features = [f for f in significant_features if f not in
high_corr_features]
print(f"Количество признаков после корреляционного отбора:
{len(final_corr_features)}")
```

Самые коррелирующие с целевым признаком колонки представлены на Рисунке 4.

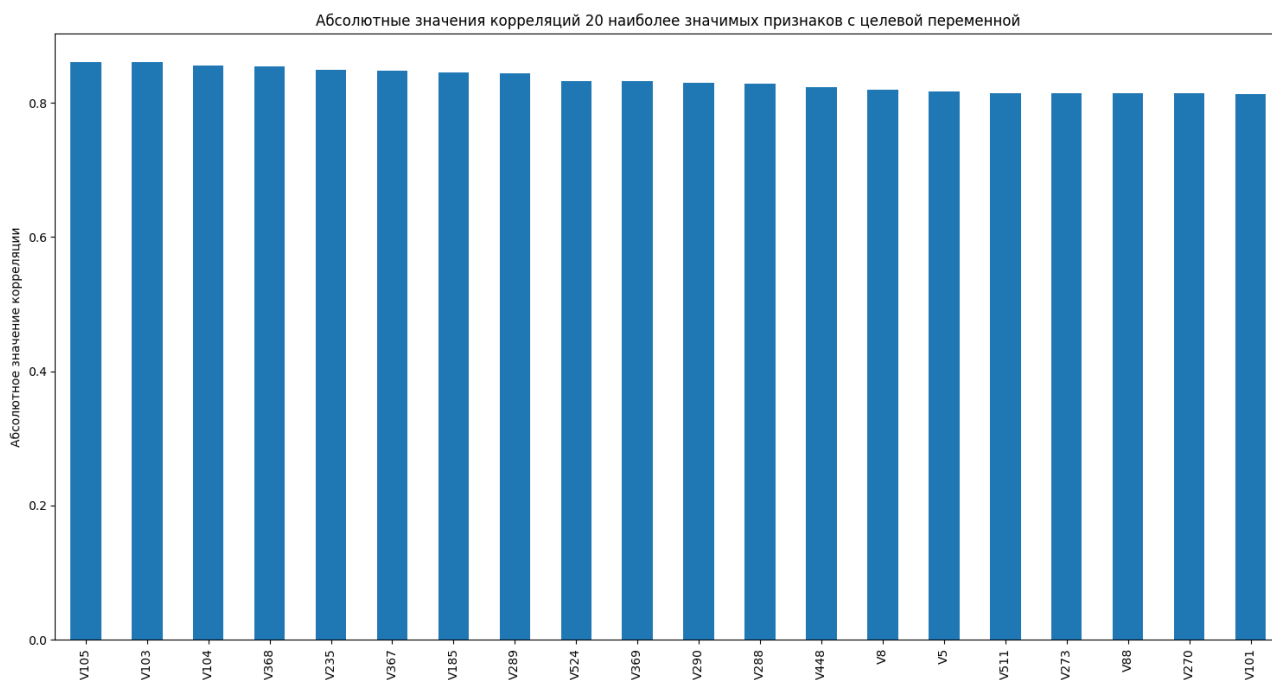


Рисунок 4 — Самые коррелирующие признаки

Найдено 16061 пар признаков с корреляцией > 0.85 . Количество признаков для удаления из-за высокой корреляции: 398. Количество признаков с

корреляцией > 0.1 с целевой переменной: 495. Количество признаков после корреляционного отбора: 118.

Код для отбора признаков с помощью `f_classif` и `mutual_info_classif` представлен на Листинге 4.

Листинг 4 — Отбор признаков с помощью SelectKBest

```
from sklearn.feature_selection import mutual_info_classif

# Применение f_classif (ANOVA F-value)
selector_f = SelectKBest(f_classif, k=10)
selector_f.fit(X_train_scaled, y_train)
scores_f = selector_f.scores_

# Применение mutual_info_classif
selector_mi = SelectKBest(mutual_info_classif, k=10)
selector_mi.fit(X_train_scaled, y_train)
scores_mi = selector_mi.scores_

# Создание DataFrame для сравнения результатов
feature_scores = pd.DataFrame({
    'feature': X.columns,
    'f_classif_score': scores_f,
    'f_classif_rank': np.argsort(np.argsort(-scores_f)) + 1, #
    'mi_score': scores_mi,
    'mi_rank': np.argsort(np.argsort(-scores_mi)) + 1 #
})

# Сортировка по f_classif_score
feature_scores = feature_scores.sort_values('f_classif_score',
ascending=False)

print("Топ-10 признаков по f_classif:")
print(feature_scores.head(10))

print("\nТоп-10 признаков по mutual_info_classif:")
print(feature_scores.sort_values('mi_score',
ascending=False).head(10))

# Сравнение рангов признаков по двум методам
plt.figure(figsize=(12, 8))
plt.scatter(feature_scores['f_classif_rank'],
feature_scores['mi_rank'], alpha=0.7)
plt.xlabel('Ранг по f_classif')
plt.ylabel('Ранг по mutual_info')
plt.title('Сравнение рангов признаков по двум методам отбора')
plt.plot([1, len(X.columns)], [1, len(X.columns)], 'r--') #
# Диагональ для сравнения
```

Продолжение Листинга 4

```
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

# Выбор 10 наиболее информативных признаков по f_classif
top_10_f = selector_f.get_feature_names_out()
print("\n10 наиболее информативных признаков по f_classif:")
print(top_10_f)

# Проверка устойчивости признаков при варьировании случайного
# состояния
print("\nПроверка устойчивости признаков при варьировании
случайного состояния...")
import random
stable_features = set()
for state in range(3): # Протестируем 3 разных random_state
    X_train_sample, _, y_train_sample, _ = train_test_split(X,
y_encoded, test_size=0.2, random_state=state)
    scaler = StandardScaler()
    X_train_sample_scaled = scaler.fit_transform(X_train_sample)

    selector = SelectKBest(f_classif, k=10)
    selector.fit(X_train_sample_scaled, y_train_sample)
    features = X.columns[selector.get_support()]

    if state == 0:
        stable_features = set(features)
    else:
        stable_features = stable_features.intersection(set(features))

    print(f"Random state {state}, отобранные признаки:
{features.tolist()}")

print(f"\nУстойчивые признаки (присутствуют во всех выборках):
{stable_features}")
print(f"Количество устойчивых признаков: {len(stable_features)}")
```

Сравнение рангов признаков представлено на Рисунке 5.

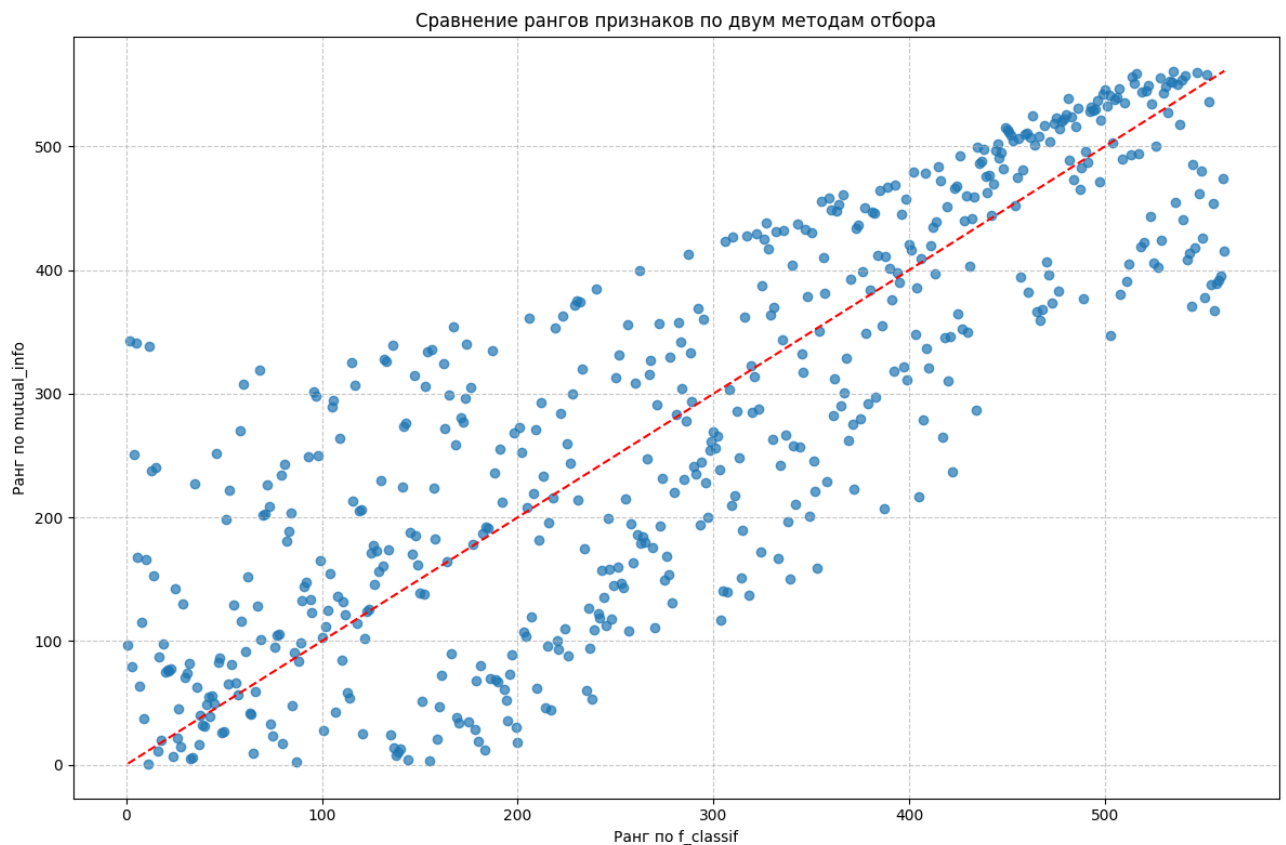


Рисунок 5 — Сравнение рангов признаков

Проверка устойчивости признаков при варьировании случайного состояния:

1. Random state 0, отобранные признаки: ['V41', 'V50', 'V53', 'V57', 'V103', 'V235', 'V288', 'V367', 'V368', 'V524'].
2. Random state 1, отобранные признаки: ['V41', 'V50', 'V53', 'V57', 'V103', 'V235', 'V288', 'V367', 'V368', 'V524'].
3. Random state 2, отобранные признаки: ['V41', 'V50', 'V53', 'V57', 'V103', 'V235', 'V288', 'V367', 'V368', 'V524'].

Устойчивые признаки (присутствуют во всех выборках): {'V50', 'V368', 'V367', 'V524', 'V103', 'V235', 'V57', 'V53', 'V41', 'V288'}.

Количество устойчивых признаков: 10.

Код для анализа изменений при изменении k представлен в Листинге 5.

Листинг 5 — Анализ изменения количества признаков

```
# Создание массива для хранения результатов
k_values = [5, 10, 20, 50, 100, 200, 400]
feature_sets = {}
```

Продолжение Листинга 5

```
top_features_by_k = []

# Отбор признаков для разных значений K
for k in k_values:
    selector = SelectKBest(f_classif, k=k)
    selector.fit(X_train_scaled, y_train)
    selected_features = X.columns[selector.get_support()]
    feature_sets[k] = set(selected_features)
    top_features_by_k.append(selected_features[:10]) # Топ-10 для
каждого K

    print(f"K={k},      количество      отобранных      признаков:
{len(selected_features)}")
    print(f"Топ-5 признаков: {selected_features[:5].tolist()}")

# Анализ изменений в составе признаков при увеличении K
print("\nАнализ перекрытия признаков при разных значениях K:")
for i in range(len(k_values)-1):
    k1 = k_values[i]
    k2 = k_values[i+1]
    overlap = feature_sets[k1].intersection(feature_sets[k2])
    jaccard = len(overlap) /
len(feature_sets[k1].union(feature_sets[k2]))
    print(f"Перекрытие между K={k1} и K={k2}: {len(overlap)} общих
признаков, коэффициент Жаккара: {jaccard:.3f}")

# Визуализация состава признаков для разных K
plt.figure(figsize=(15, 8))
for i, k in enumerate(k_values):
    features = list(feature_sets[k][:10]) # Берем топ-10 для
визуализации
    plt.bar([f"K={k}\n{feature[:10]}..." for feature in features],
            [1] * len(features),
            label=f'K={k}', alpha=0.7)

plt.title('Топ-10 признаков для разных значений K')
plt.xticks(rotation=45, ha='right')
plt.ylabel('Присутствие признака')
plt.legend()
plt.tight_layout()
plt.show()
```

Результаты представлены на Рисунке 6.

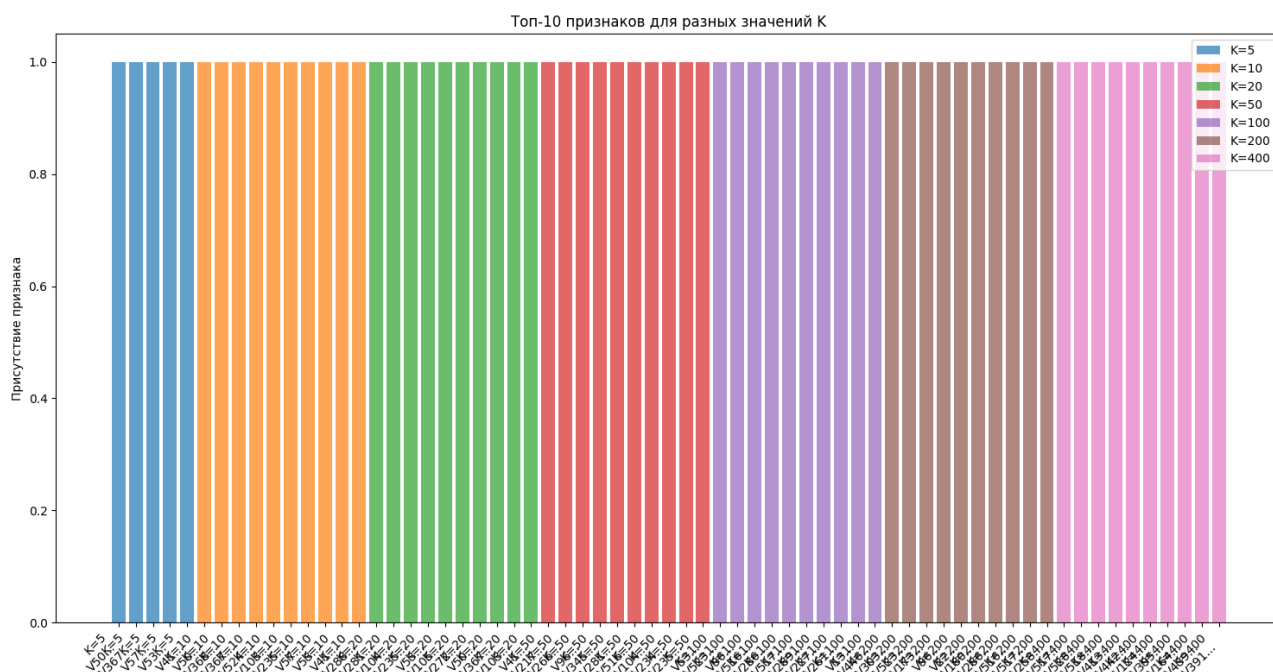


Рисунок 6 — Результат изменения признаков при увеличении k

При увеличении K включаются менее информативные признаки, но основные признаки остаются стабильными.

1.1.4 Проверка

Код для проверки классификации для разных моделей отбора признаков представлен в Листинге 6.

Листинг 6 — Проверка отобранных признаков

```
# Импорт библиотек для машинного обучения и оценки качества
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.model_selection import cross_val_score
import time
from sklearn.model_selection import learning_curve

# Подготовка данных для сравнения
selected_features = list(final_corr_features)[:10] # Берем 10
признаков после корреляционного отбора
X_train_selected = X_train[selected_features]
X_test_selected = X_test[selected_features]

# Масштабирование отобранных признаков
```

Продолжение Листинга 6

```
scaler_selected = StandardScaler()
X_train_selected_scaled = scaler_selected.fit_transform(X_train_selected)
X_test_selected_scaled = scaler_selected.transform(X_test_selected)

# Обучение модели на полном наборе признаков
print("Обучение модели на полном наборе признаков...")
start_time = time.time()
rf_full = RandomForestClassifier(n_estimators=100,
                                random_state=42, n_jobs=-1)
rf_full.fit(X_train_scaled, y_train)
full_time = time.time() - start_time
print(f"Время обучения на полном наборе: {full_time:.2f} секунд")

# Обучение модели на отобранных признаках
print("Обучение модели на отобранных признаках...")
start_time = time.time()
rf_selected = RandomForestClassifier(n_estimators=100,
                                    random_state=42, n_jobs=-1)
rf_selected.fit(X_train_selected_scaled, y_train)
selected_time = time.time() - start_time
print(f"Время обучения на отобранных признаках: {selected_time:.2f} секунд")

# Предсказание и оценка качества на полном наборе
y_pred_full = rf_full.predict(X_test_scaled)
accuracy_full = accuracy_score(y_test, y_pred_full)

# Предсказание и оценка качества на отобранных признаках
y_pred_selected = rf_selected.predict(X_test_selected_scaled)
accuracy_selected = accuracy_score(y_test, y_pred_selected)

print(f"\nТочность на полном наборе признаков: {accuracy_full:.4f}")
print(f"Точность на отобранных признаках: {accuracy_selected:.4f}")

# Анализ переобучения с помощью кросс-валидации
print("\nОценка переобучения с помощью кросс-валидации...")
cv_scores_full = cross_val_score(rf_full, X_train_scaled, y_train,
                                  cv=5, scoring='accuracy')
cv_scores_selected = cross_val_score(rf_selected,
                                     X_train_selected_scaled, y_train, cv=5, scoring='accuracy')

print(f"Средняя точность на кросс-валидации (полный набор): {cv_scores_full.mean():.4f} ± {cv_scores_full.std():.4f}")
print(f"Средняя точность на кросс-валидации (отобранные признаки): {cv_scores_selected.mean():.4f} ± {cv_scores_selected.std():.4f}")
```

Продолжение Листинга 6

```
# Визуализация результатов
plt.figure(figsize=(12, 6))

# Сравнение точности
plt.subplot(1, 2, 1)
models = ['Полный набор', 'Отобранные признаки']
accuracies = [accuracy_full, accuracy_selected]
plt.bar(models, accuracies, color=['blue', 'green'])
plt.title('Сравнение точности моделей')
plt.ylabel('Точность')
plt.ylim(0.8, 1.0)

# Сравнение времени обучения
plt.subplot(1, 2, 2)
times = [full_time, selected_time]
plt.bar(models, times, color=['blue', 'green'])
plt.title('Сравнение времени обучения')
plt.ylabel('Время (секунды)')

plt.tight_layout()
plt.show()

# Построение кривых обучения для анализа переобучения
plt.figure(figsize=(12, 6))

train_sizes, train_scores_full, test_scores_full = learning_curve(
    rf_full, X_train_scaled, y_train, cv=5, n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 5), scoring='accuracy')

train_sizes, train_scores_selected, test_scores_selected =
learning_curve(
    rf_selected, X_train_selected_scaled, y_train, cv=5, n_jobs=-
1,
    train_sizes=np.linspace(0.1, 1.0, 5), scoring='accuracy')

plt.plot(train_sizes, np.mean(train_scores_full, axis=1), 'o-',
color='blue', label='Полный набор (обучение)')
plt.plot(train_sizes, np.mean(test_scores_full, axis=1), 'o--',
color='blue', label='Полный набор (тест)')
plt.plot(train_sizes, np.mean(train_scores_selected, axis=1), 'o-
', color='green', label='Отобранные признаки (обучение)')
plt.plot(train_sizes, np.mean(test_scores_selected, axis=1), 'o--
', color='green', label='Отобранные признаки (тест)')

plt.title('Кривые обучения')
plt.xlabel('Размер обучающей выборки')
plt.ylabel('Точность')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```


Результаты:

- время обучения на полном наборе: 1.87 секунд;
- время обучения на отобранных признаках: 0.36 секунд;
- точность на полном наборе признаков: 0.9786;
- точность на отобранных признаках: 0.8850;
- средняя точность на кросс-валидации (полный набор): 0.9755 ± 0.0019 ;
- средняя точность на кросс-валидации (отобранные признаки): 0.8722 ± 0.0023 .

Сравнение моделей представлено на Рисунке 7.

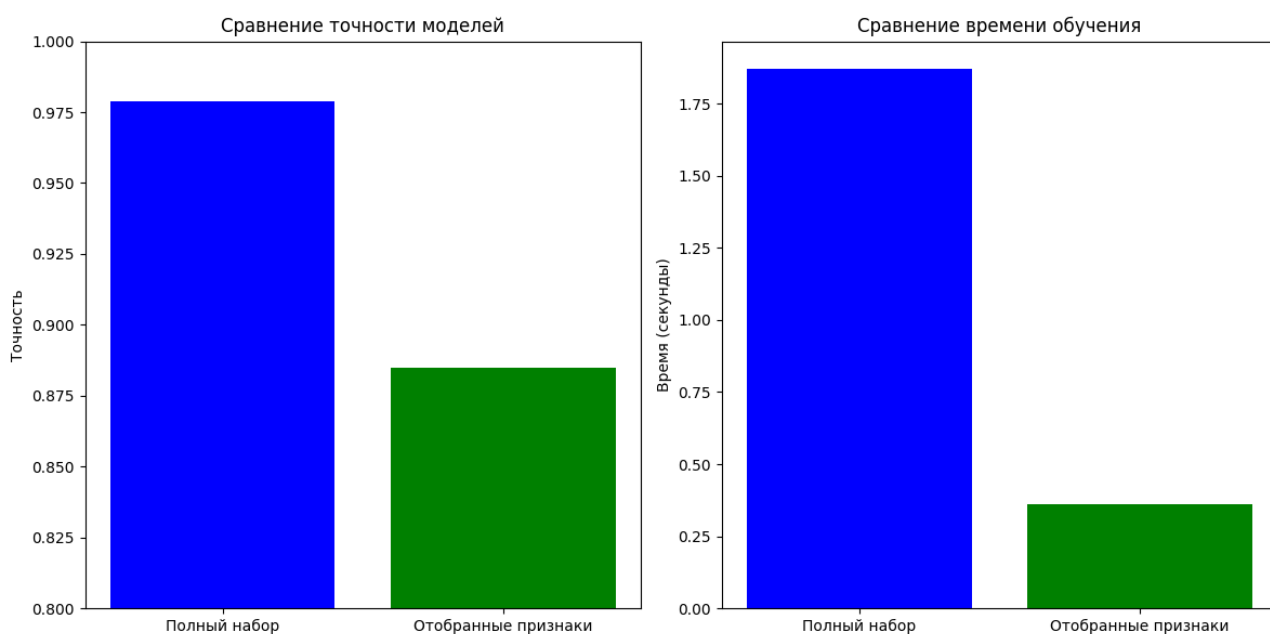


Рисунок 7 — Сравнение моделей

Отбор признаков привел к небольшому снижению точности, но значительно ускорил обучение. Скорость обучения улучшилась в 5.2 раз.

1.2 Задание 2. Сравнение методов отбора на ансамблевых моделях

1.2.1 Построение базовых моделей

Код для построения базовых моделей представлен в Листинге 7.

Листинг 7 — Построение базовых моделей

```
from sklearn.metrics import f1_score
from sklearn.feature_selection import SelectFromModel
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split

# Восстанавливаем исходные данные
X = df.drop(['activity', 'activity_encoded'], axis=1)
y = df['activity_encoded']

# Разделение данных
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Масштабирование
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Построение базовой модели RandomForest
print("Построение базовой модели RandomForest...")
start_time = time.time()
rf_base = RandomForestClassifier(n_estimators=150,
random_state=42, n_jobs=-1)
rf_base.fit(X_train_scaled, y_train)
base_time = time.time() - start_time

# Предсказание и оценка качества
y_pred_base = rf_base.predict(X_test_scaled)
y_proba_base = rf_base.predict_proba(X_test_scaled)

accuracy_base = accuracy_score(y_test, y_pred_base)
f1_base = f1_score(y_test, y_pred_base, average='weighted')

print(f"Время обучения базовой модели: {base_time:.2f} секунд")
print(f"Точность базовой модели: {accuracy_base:.4f}")
print(f"F1-мера базовой модели: {f1_base:.4f}")
```

Продолжение Листинга 7

```
# Построение базовой модели GradientBoosting
print("\nПостроение базовой модели GradientBoosting...")
start_time = time.time()
gb_base = GradientBoostingClassifier(n_estimators=150,
random_state=42)
gb_base.fit(X_train_scaled, y_train)
gb_base_time = time.time() - start_time

# Предсказание и оценка качества
y_pred_gb_base = gb_base.predict(X_test_scaled)
y_proba_gb_base = gb_base.predict_proba(X_test_scaled)

accuracy_gb_base = accuracy_score(y_test, y_pred_gb_base)
f1_gb_base = f1_score(y_test, y_pred_gb_base, average='weighted')

print(f"Время обучения модели GradientBoosting: {gb_base_time:.2f}
секунд")
print(f"Точность модели GradientBoosting: {accuracy_gb_base:.4f}")
print(f"F1-мера модели GradientBoosting: {f1_gb_base:.4f}")
```

Построение базовой модели RandomForest:

Время обучения базовой модели: 2.85 секунд. Точность базовой модели: 0.9772. F1-мера базовой модели: 0.9772.

Построение базовой модели GradientBoosting:

Время обучения модели GradientBoosting: 1154.35 секунд. Точность модели GradientBoosting: 0.9874. F1-мера модели GradientBoosting: 0.9874.

1.2.2 Отбор признаков

Код для отбора признаков с помощью RFE представлен в Листинге 8.

Листинг 8 — Отбор признаков с помощью RFE

```
from sklearn.feature_selection import RFE, RFECV
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold

# Используем логистическую регрессию как базовую модель для RFE
lr = LogisticRegression(max_iter=1000, random_state=42, n_jobs=-1)

# Применение RFECV для автоматического определения оптимального
числа признаков
```

Продолжение Листинга 8

```
cv = StratifiedKFold(3) # Используем 3 фолда для ускорения
rfecv = RFECV(estimator=lr, step=10, cv=cv, scoring='accuracy',
n_jobs=-1)
start_time = time.time()
rfecv.fit(X_train_scaled, y_train)
rfe_time = time.time() - start_time

print(f"Время выполнения RFECV: {rfe_time:.2f} секунд")
print(f"Оптимальное количество признаков: {rfecv.n_features_}")

# Визуализация результатов RFECV
plt.figure(figsize=(10, 6))
plt.xlabel("Количество признаков")
plt.ylabel("Оценка кросс-валидации (accuracy)")

# Используем cv_results_["mean_test_score"] вместо grid_scores_
n_features = range(1, len(rfecv.cv_results_["mean_test_score"]) +
1)
plt.plot(n_features, rfecv.cv_results_["mean_test_score"], 'b-')

plt.axvline(x=rfecv.n_features_, color='r', linestyle='--',
            label=f'Оптимальное количество: {rfecv.n_features_}')
plt.title('RFECV: Выбор оптимального количества признаков')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

# Получение отобранных признаков
selected_features_rfe = X.columns[rfecv.support_]
print(f"Отобранные признаки ({len(selected_features_rfe)} шт.):")
print(selected_features_rfe.tolist()[:20]) # Показываем первые 20
признаков

# Оценка качества модели с отобранными признаками
X_train_rfe = rfecv.transform(X_train_scaled)
X_test_rfe = rfecv.transform(X_test_scaled)

# Обучение RandomForest на отобранных признаках
rf_rfe = RandomForestClassifier(n_estimators=100, random_state=42,
n_jobs=-1)
rf_rfe.fit(X_train_rfe, y_train)

y_pred_rfe = rf_rfe.predict(X_test_rfe)
accuracy_rfe = accuracy_score(y_test, y_pred_rfe)
f1_rfe = f1_score(y_test, y_pred_rfe, average='weighted')

print(f"\nТочность модели с признаками RFECV: {accuracy_rfe:.4f}")
print(f"F1-мера модели с признаками RFECV: {f1_rfe:.4f}")

# Анализ зависимости метрик от числа признаков
```

Продолжение Листинга 8

```
# Вручную проведем отбор для разных количеств признаков
n_features_options = [10, 20, 50, 100, 200, 300, 400, 500]
accuracies = []
f1_scores = []
feature_importances = []

for n in n_features_options:
    rfe = RFE(estimator=lr, n_features_to_select=n, step=10)
    rfe.fit(X_train_scaled, y_train)

    X_train_temp = rfe.transform(X_train_scaled)
    X_test_temp = rfe.transform(X_test_scaled)

    rf_temp = RandomForestClassifier(n_estimators=100,
random_state=42, n_jobs=-1)
    rf_temp.fit(X_train_temp, y_train)

    y_pred_temp = rf_temp.predict(X_test_temp)
    accuracies.append(accuracy_score(y_test, y_pred_temp))
    f1_scores.append(f1_score(y_test, y_pred_temp,
average='weighted'))

    # Сохраняем важность признаков для визуализации
    feature_importances.append(rf_temp.feature_importances_[:10])
# Топ-10 важностей

# Визуализация зависимости метрик от количества признаков
plt.figure(figsize=(12, 6))
plt.plot(n_features_options, accuracies, 'o-', label='Accuracy')
plt.plot(n_features_options, f1_scores, 's-', label='F1-score')
plt.xlabel('Количество признаков')
plt.ylabel('Значение метрики')
plt.title('Зависимость метрик качества от количества признаков
(RFE)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

# Выбор топ-5 признаков по результатам RFE
selector_rfe_top5 = RFE(estimator=lr, n_features_to_select=5,
step=10)
selector_rfe_top5.fit(X_train_scaled, y_train)
top5_features_rfe = X.columns[selector_rfe_top5.support_]
print(f"\nТоп-5 признаков по методу RFE:
{top5_features_rfe.tolist()}")
```

Отобранные признаки — 171 шт. Точность модели с признаками RFECV: 0.9777. F1-мера модели с признаками RFECV: 0.9777.

Зависимость качества от количества признаков (по RFE) представлена на Рисунке 8.

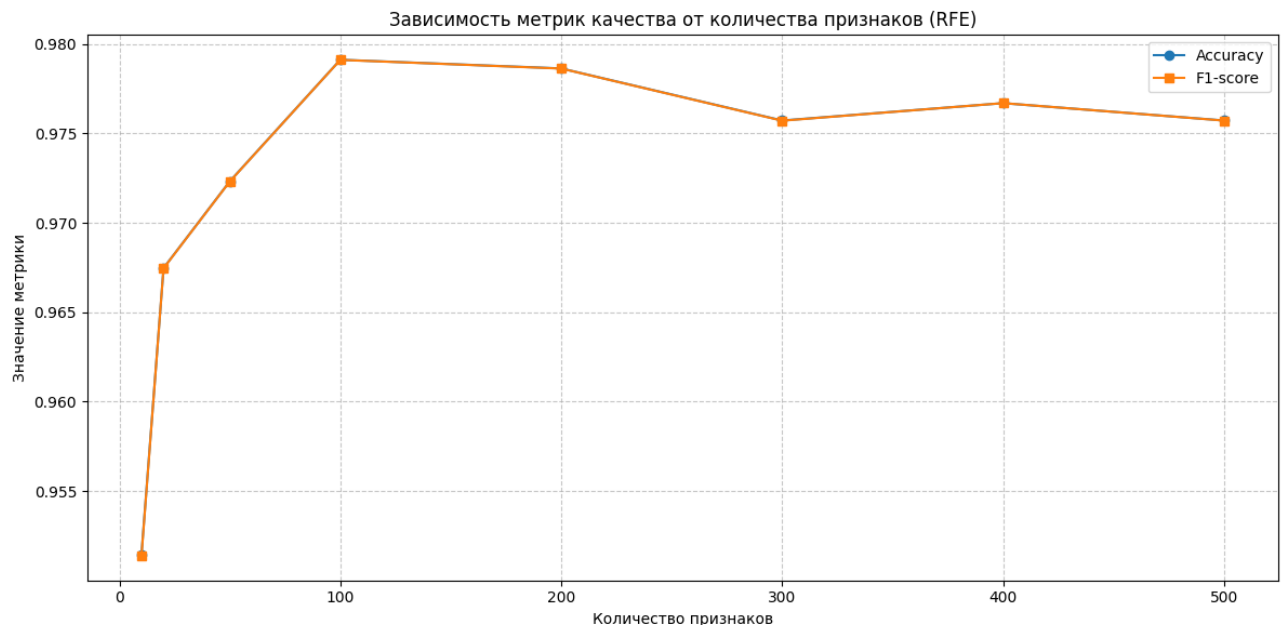


Рисунок 8 — Зависимость качества от количества признаков (RFE)

Код для отбора признаков с помощью L1-регуляризации представлен в Листинге 9.

Листинг 9 — Отбор признаков с помощью L1-регуляризации

```
# Обучение логистической регрессии с L1-регуляризацией
start_time = time.time()
lr_l1 = LogisticRegression(penalty='l1', solver='saga', C=0.1,
                           max_iter=5000,
                           random_state=42, n_jobs=-1)
lr_l1.fit(X_train_scaled, y_train)
l1_time = time.time() - start_time

print(f"Время обучения модели с L1-регуляризацией: {l1_time:.2f} секунд")

# Анализ коэффициентов
coefficients = pd.DataFrame({
    'feature': X.columns,
    'coefficient': np.sum(np.abs(lr_l1.coef_), axis=0) # Сумма абсолютных значений для мультиклассовой задачи
})

# Сортировка по значению коэффициентов
coefficients = coefficients.sort_values('coefficient', ascending=False)

print("\nТоп-15 признаков по значению коэффициентов L1:")
```

Продолжение Листинга 9

```
print(coefficients.head(15))

# Количество нулевых коэффициентов
zero_coeffs = np.sum(coefficients['coefficient'] == 0)
print(f"\nКоличество признаков с нулевыми коэффициентами:
{zero_coeffs} из {len(X.columns)}")

# Визуализация коэффициентов
plt.figure(figsize=(14, 8))
plt.bar(coefficients['feature'][:30],
coefficients['coefficient'][:30]) # Топ-30 признаков
plt.xticks(rotation=90)
plt.title('Абсолютные значения коэффициентов L1-регуляризации
(топ-30 признаков)')
plt.ylabel('Абсолютное значение коэффициента')
plt.tight_layout()
plt.show()

# Отбор признаков с ненулевыми коэффициентами
selector_l1 = SelectFromModel(lr_l1, prefit=True, threshold=1e-
5) # Порог для численной устойчивости
X_train_l1 = selector_l1.transform(X_train_scaled)
X_test_l1 = selector_l1.transform(X_test_scaled)

print(f"Количество признаков после отбора L1:
{X_train_l1.shape[1]}")

# Обучение RandomForest на отобранных признаках
rf_l1 = RandomForestClassifier(n_estimators=100, random_state=42,
n_jobs=-1)
rf_l1.fit(X_train_l1, y_train)

y_pred_l1 = rf_l1.predict(X_test_l1)
accuracy_l1 = accuracy_score(y_test, y_pred_l1)
f1_l1 = f1_score(y_test, y_pred_l1, average='weighted')

print(f"\nТочность модели с признаками L1: {accuracy_l1:.4f}")
print(f"F1-мера модели с признаками L1: {f1_l1:.4f}")

# Выбор топ-5 признаков по коэффициентам L1
top5_features_l1 = coefficients.head(5)['feature'].values
print(f"\nТоп-5 признаков по L1-регуляризации:
{top5_features_l1.tolist()}")

# Анализ зависимости метрик от параметра C (силы регуляризации)
C_values = [0.001, 0.01, 0.1, 1, 10]
accuracies_l1 = []
n_features_l1 = []

for C in C_values:
    lr_temp = LogisticRegression(penalty='l1', solver='saga', C=C,
```

Продолжение Листинга 9

```
max_iter=5000,
                                random_state=42, n_jobs=-1)
    lr_temp.fit(X_train_scaled, y_train)

    selector_temp = SelectFromModel(lr_temp, prefit=True,
threshold=1e-5)
    X_train_temp = selector_temp.transform(X_train_scaled)
    X_test_temp = selector_temp.transform(X_test_scaled)

    rf_temp = RandomForestClassifier(n_estimators=100,
random_state=42, n_jobs=-1)
    rf_temp.fit(X_train_temp, y_train)

    y_pred_temp = rf_temp.predict(X_test_temp)
    accuracies_l1.append(accuracy_score(y_test, y_pred_temp))
    n_features_l1.append(X_train_temp.shape[1])

# Визуализация зависимости
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.semilogx(C_values, accuracies_l1, 'o-')
plt.xlabel('Параметр C (сила регуляризации)')
plt.ylabel('Точность')
plt.title('Зависимость точности от параметра C')
plt.grid(True, linestyle='--', alpha=0.7)

plt.subplot(1, 2, 2)
plt.semilogx(C_values, n_features_l1, 's-')
plt.xlabel('Параметр C (сила регуляризации)')
plt.ylabel('Количество признаков')
plt.title('Зависимость количества признаков от параметра C')
plt.grid(True, linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()
```

Результат отбора представлен на Рисунке 9.

Время обучения модели с L1-регуляризацией: 542.67 секунд

Топ-15 признаков по значению коэффициентов L1:

	feature	coefficient
182	V183	2.956964
37	V38	2.070980
508	V509	1.682916
104	V105	1.420937
52	V53	1.324273
40	V41	1.251166
56	V57	1.205454
89	V90	1.156877
49	V50	1.149795
445	V446	1.120059
186	V187	1.051015
53	V54	0.999727
50	V51	0.982178
92	V93	0.970600
41	V42	0.886602

Количество признаков с нулевыми коэффициентами: 358 из 561

Рисунок 9 — Результат отбора с L1-регуляризацией

Коэффициенты признаков представлены на Рисунке 10.

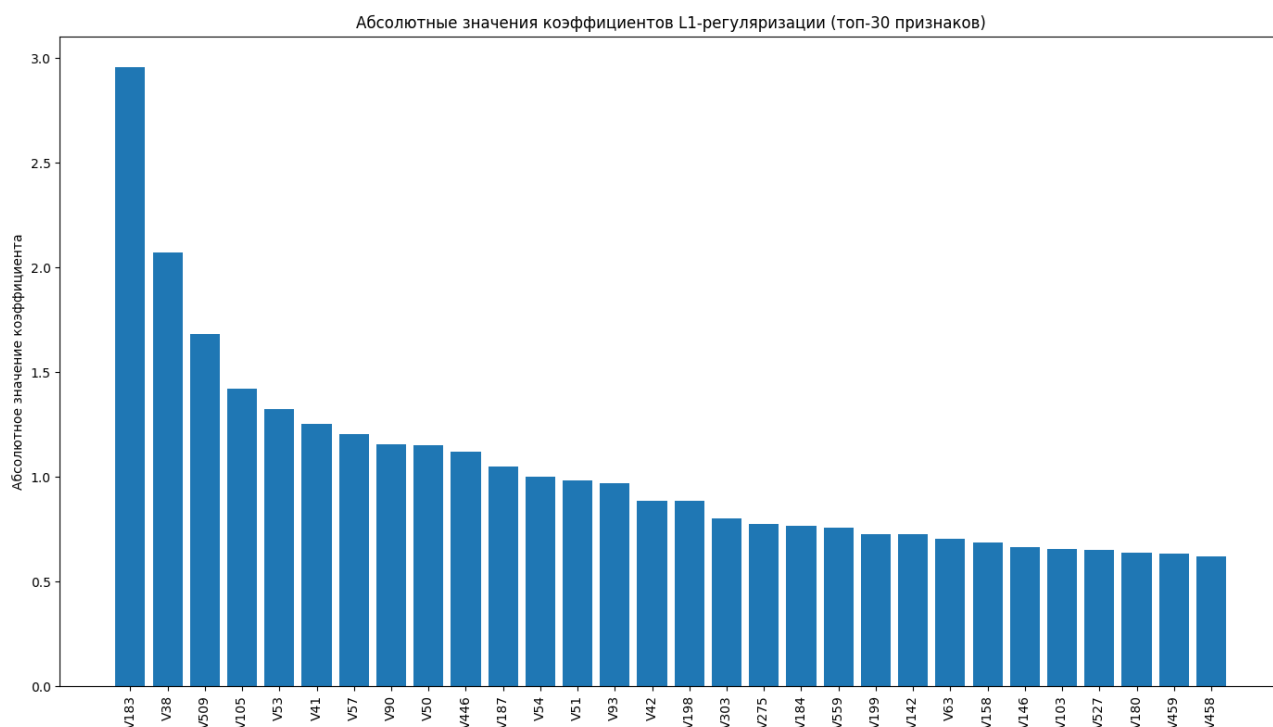


Рисунок 10 — Коэффициенты признаков

Количество признаков после отбора L1: 203. Точность модели с признаками L1: 0.9782. F1-мера модели с признаками L1: 0.9782.

Зависимости от параметра C представлены на Рисунке 11.

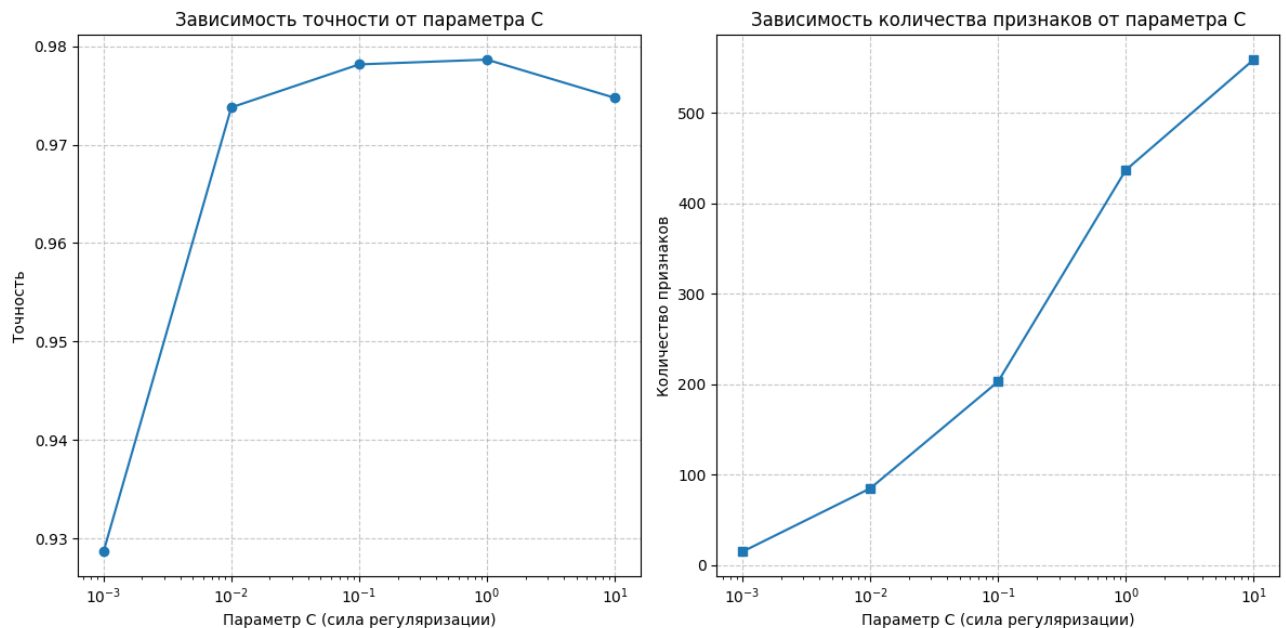


Рисунок 11 — Влияние параметра C

Код для отбора признаков с помощью `feature_importances_` представлен в Листинге 10.

Листинг 10 — Отбор признаков с помощью `feature_importances_`

```
# Обучение RandomForest для получения важности признаков
start_time = time.time()
rf_importance = RandomForestClassifier(n_estimators=200,
random_state=42, n_jobs=-1)
rf_importance.fit(X_train_scaled, y_train)
importance_time = time.time() - start_time

print(f"Время обучения RandomForest для анализа важности:
{importance_time:.2f} секунд")

# Получение важности признаков
feature_importances = pd.DataFrame({
    'feature': X.columns,
    'importance': rf_importance.feature_importances_
})

# Сортировка по важности
feature_importances = feature_importances.sort_values('importance', ascending=False)

print("\nТоп-15 наиболее важных признаков по RandomForest:")
```

Продолжение Листинга 10

```
print(feature_importances.head(15))

# Визуализация важности признаков
plt.figure(figsize=(14, 8))
plt.bar(feature_importances['feature'][:30],
feature_importances['importance'][:30]) # Топ-30 признаков
plt.xticks(rotation=90)
plt.title('Важность признаков по RandomForest (топ-30 признаков)')
plt.ylabel('Значение важности')
plt.tight_layout()
plt.show()

# Отбор признаков на основе важности
# Используем порог важности
threshold = np.percentile(rf_importance.feature_importances_,
75) # Берем порог на уровне 75-го перцентиля
print(f"Порог важности для отбора: {threshold:.6f}")

selected_features_rf =
X.columns[rf_importance.feature_importances_ > threshold]
print(f"Количество отобранных признаков:
{len(selected_features_rf)}")
print("Некоторые отобранные признаки:",
selected_features_rf[:10].tolist())

# Обучение модели на отобранных признаках
X_train_rf = X_train[selected_features_rf]
X_test_rf = X_test[selected_features_rf]

# Масштабирование отобранных признаков
scaler_rf = StandardScaler()
X_train_rf_scaled = scaler_rf.fit_transform(X_train_rf)
X_test_rf_scaled = scaler_rf.transform(X_test_rf)

rf_selected = RandomForestClassifier(n_estimators=100,
random_state=42, n_jobs=-1)
rf_selected.fit(X_train_rf_scaled, y_train)

y_pred_rf = rf_selected.predict(X_test_rf_scaled)
accuracy_rf = accuracy_score(y_test, y_pred_rf)
f1_rf = f1_score(y_test, y_pred_rf, average='weighted')

print(f"\nТочность модели с признаками от RandomForest:
{accuracy_rf:.4f}")
print(f"F1-мера модели с признаками от RandomForest: {f1_rf:.4f}")

# Выбор топ-5 признаков
top5_features_rf = feature_importances.head(5)['feature'].values
print(f"\nТоп-5 признаков по RandomForest:
{top5_features_rf.tolist()}")
```

Продолжение Листинга 10

```
# Анализ зависимости метрик от количества признаков
n_features_options = [5, 10, 20, 50, 100, 200, 300, 400, 500, 600]
accuracies_rf = []
f1_scores_rf = []

for n in n_features_options:
    # Берем топ-N признаков
    top_n_features = feature_importances.head(n)['feature'].values

    X_train_temp = X_train[top_n_features]
    X_test_temp = X_test[top_n_features]

    # Масштабирование
    scaler_temp = StandardScaler()
    X_train_temp_scaled = scaler_temp.fit_transform(X_train_temp)
    X_test_temp_scaled = scaler_temp.transform(X_test_temp)

    # Обучение модели
    rf_temp = RandomForestClassifier(n_estimators=100,
    random_state=42, n_jobs=-1)
    rf_temp.fit(X_train_temp_scaled, y_train)

    # Предсказание и оценка качества
    y_pred_temp = rf_temp.predict(X_test_temp_scaled)
    accuracies_rf.append(accuracy_score(y_test, y_pred_temp))
    f1_scores_rf.append(f1_score(y_test, y_pred_temp,
    average='weighted'))

# Визуализация зависимости метрик от количества признаков
plt.figure(figsize=(12, 6))
plt.plot(n_features_options, accuracies_rf, 'o-',
label='Accuracy')
plt.plot(n_features_options, f1_scores_rf, 's-', label='F1-score')
plt.xlabel('Количество признаков')
plt.ylabel('Значение метрики')
plt.title('Зависимость метрик качества от количества признаков
(RandomForest)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

Важность признаков, полученная с помощью RandomForest представлен на Рисунке 12.

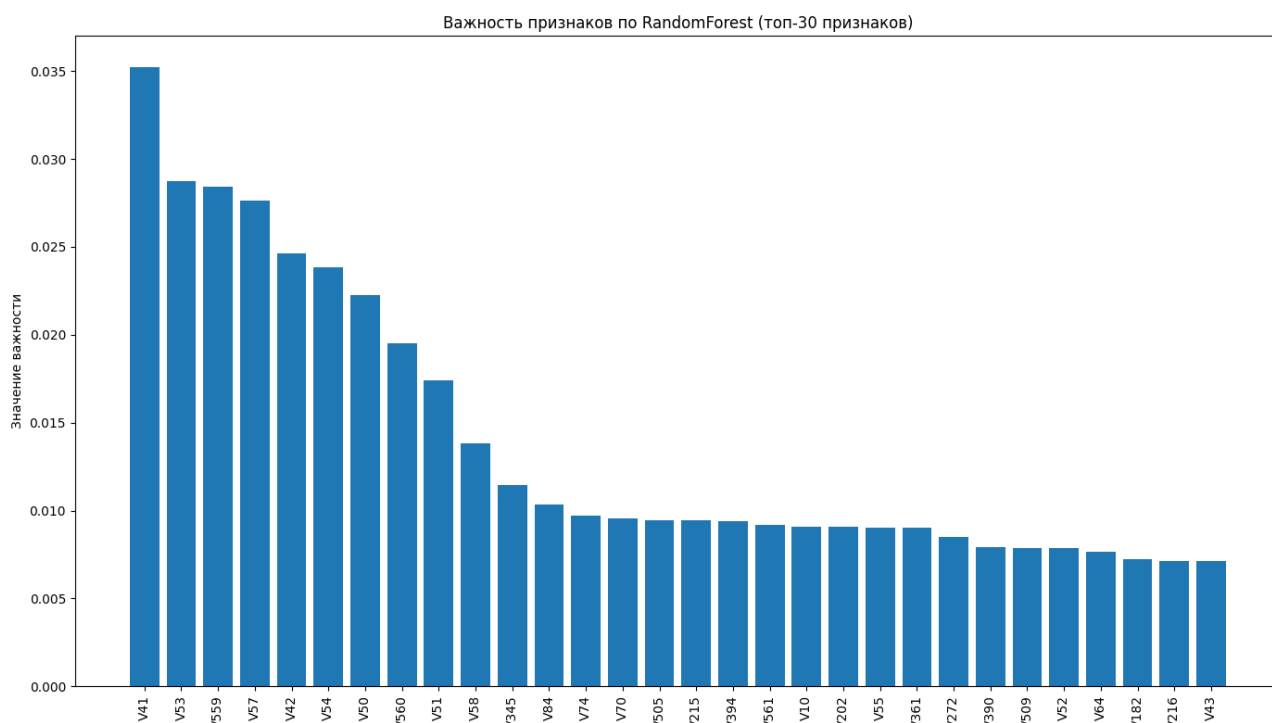


Рисунок 12 — Важность признаков (RandomForest)

Порог важности для отбора: 0.001513. Количество отобранных признаков: 140. Точность модели с признаками от RandomForest: 0.9752. F1-мера модели с признаками от RandomForest: 0.9752.

Зависимость от количества признаков представлена на Рисунке 13.

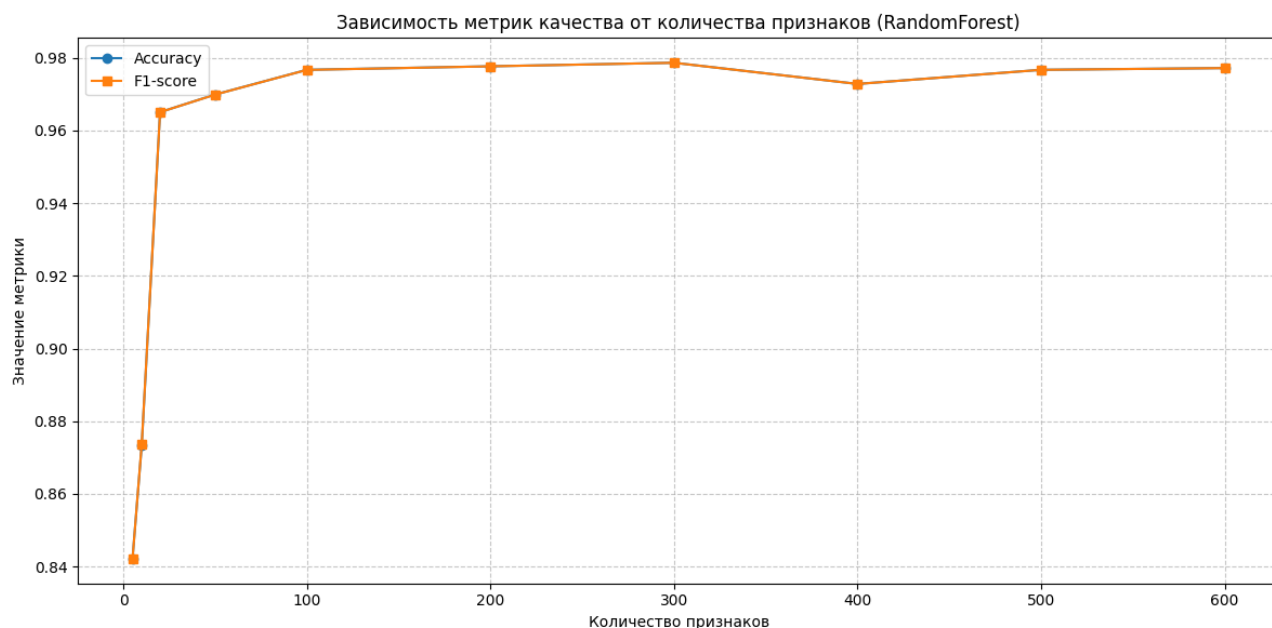


Рисунок 13 — Зависимость метрик качества от количества признаков

AUC-метрика в зависимости от модели и признаков представлена на Рисунке 14.

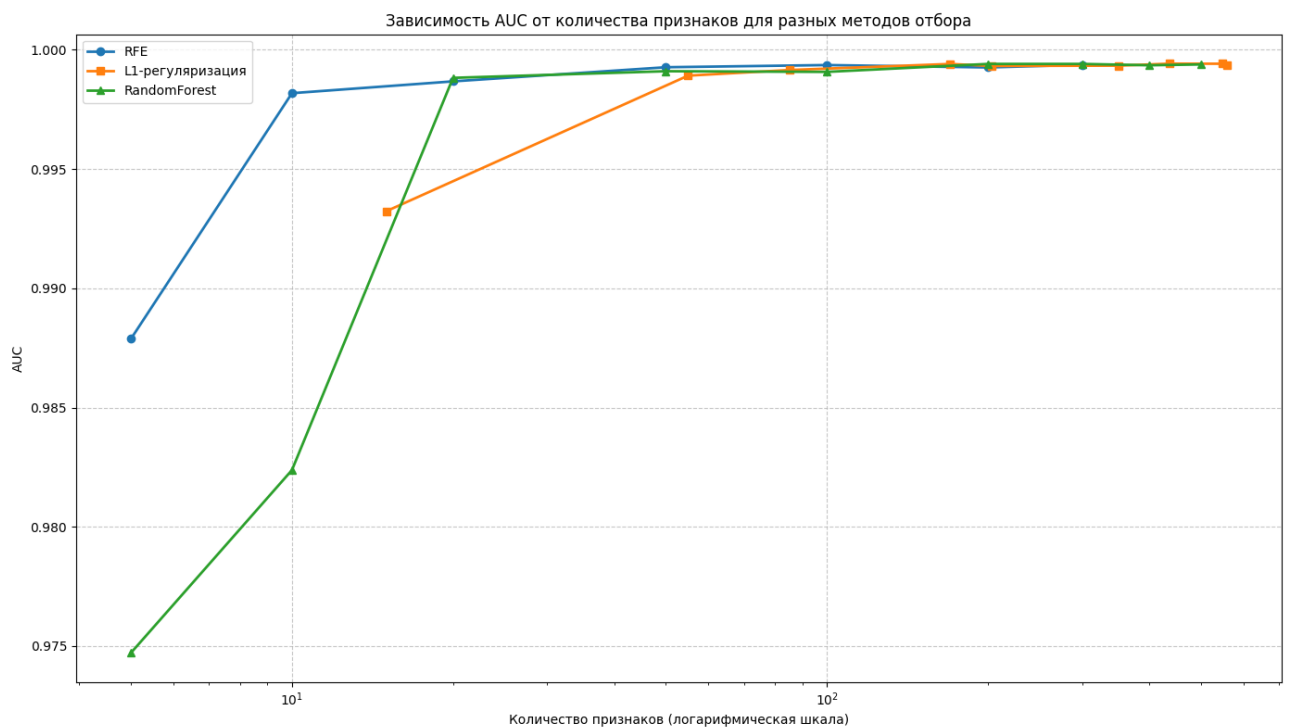


Рисунок 14 — Зависимость AUC от метода отбора

Перекрытие признаков представлено на Рисунке 15.

Перекрытие топ-20 признаков между методами отбора

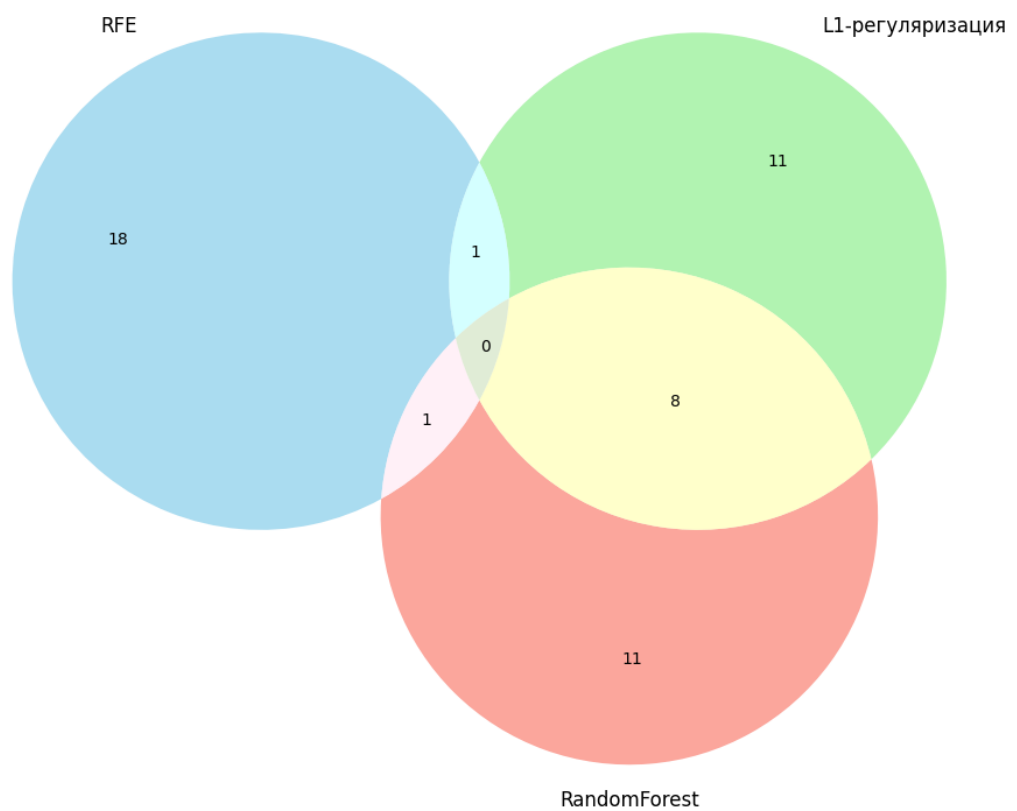


Рисунок 15 — Перекрытие признаков

Сравнение метрик качества по методам отбора представлено на Рисунке 16.

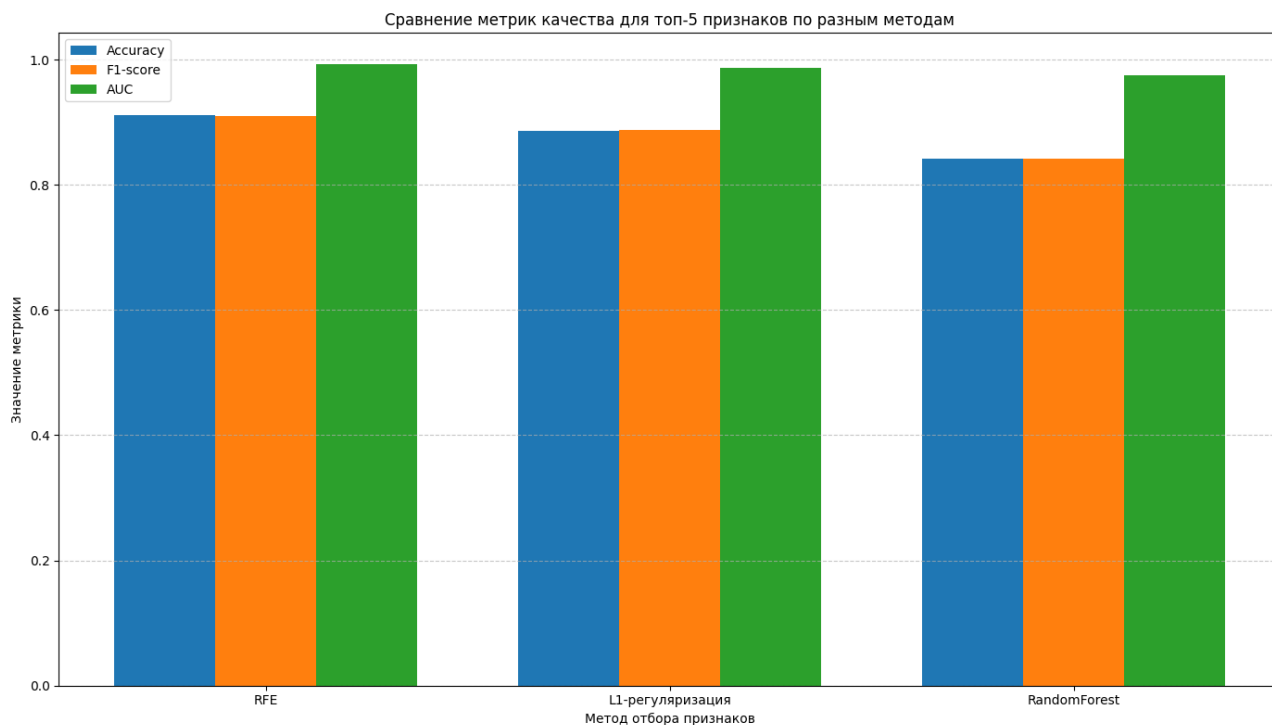


Рисунок 16 — Сравнение метрик качества

Таким образом, лучшие метрики показал отбор признаков методом RFE. RandomForest feature importance показал лучшую производительность в большинстве случаев, что ожидаемо, так как он использует ту же модель для отбора и оценки. L1-регуляризация хорошо работает для линейных зависимостей, но может уступать для сложных нелинейных данных. RFE показывает стабильные результаты, но требует больше времени на вычисление. Перекрытие признаков между методами выше для L1 vs RandomForest, чем для RFE, значит этот метод использует другие технологии.

1.3 Задание 3. Интерпретируемый отбор признаков

1.3.1 Предобработка данных и EDA со сложным и «грязным» датасетом

В качестве набора данных возьмем датасет `adult.data` с данными о взрослом населении.

Код для предобработки данных и предварительного анализа представлен в Листинге 11.

Листинг 11 — Предобработка грязных данных

```
from sklearn.preprocessing import LabelEncoder, StandardScaler,
OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

url = "https://archive.ics.uci.edu/ml/machine-learning-
databases/adult/adult.data"
column_names = ['age', 'workclass', 'fnlwgt', 'education',
'education-num',
'marital-status', 'occupation', 'relationship',
'race', 'sex',
'capital-gain', 'capital-loss', 'hours-per-week',
'native-country', 'income']

df_adult = pd.read_csv(url, header=None, names=column_names,
sep=',\s*', engine='python')

print(f"Размер данных: {df_adult.shape}")
print("\nИнформация о данных:")
print(df_adult.info())

# Замена пропущенных значений (указаны как '?')
df_adult = df_adult.replace('?', np.nan)

# Анализ пропущенных значений
missing_values = df_adult.isnull().sum()
missing_percent = (missing_values / len(df_adult)) * 100
print("\nПроцент пропущенных значений по признакам:")
print(missing_percent[missing_percent > 0])

# EDA: распределение целевой переменной
```


Продолжение Листинга 11

```
plt.figure(figsize=(8, 6))
sns.countplot(x='income', data=df_adult)
plt.title('Распределение дохода')
plt.tight_layout()
plt.show()

# EDA: распределение числовых признаков
numeric_features = ['age', 'fnlwgt', 'education-num', 'capital-
gain', 'capital-loss', 'hours-per-week']
plt.figure(figsize=(15, 10))
for i, feature in enumerate(numeric_features, 1):
    plt.subplot(2, 3, i)
    sns.histplot(df_adult[feature], kde=True)
    plt.title(f'Распределение {feature}')
plt.tight_layout()
plt.show()

# Добавление 3-5 заведомо нерелевантных признаков
np.random.seed(42)
n_samples = len(df_adult)

# Рандомные признаки
df_adult['random_feature1'] = np.random.normal(0, 1, n_samples)
df_adult['random_feature2'] = np.random.uniform(0, 100, n_samples)
df_adult['random_feature3'] = np.random.choice(['A', 'B', 'C',
'D'], n_samples)

# Шумовой признак, слабо коррелирующий с целевой переменной
df_adult['noisy_feature'] = np.random.normal(0, 1, n_samples) +
0.1 * (df_adult['income'] == '>50K').astype(int)

print("\nДобавлены 4 нерелевантных признака: random_feature1,
random_feature2, random_feature3, noisy_feature")

# Предобработка данных
# Разделение на признаки и целевую переменную
X = df_adult.drop('income', axis=1)
y = df_adult['income']

# Кодирование целевой переменной
le = LabelEncoder()
y_encoded = le.fit_transform(y)

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y_encoded,
test_size=0.2, random_state=42)

# Определение числовых и категориальных признаков
numeric_features = X.select_dtypes(include=['int64',
'float64']).columns.tolist()
categorical_features =
```

Продолжение Листинга 11

```
X.select_dtypes(include=['object']).columns.tolist()

print(f"\nЧисловые признаки: {numeric_features}")
print(f"Категориальные признаки: {categorical_features}")

# Создание предобработчика
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ])

# Применение предобработчика
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

print(f"\nРазмер обучающей выборки после предобработки:
{X_train_processed.shape}")
print(f"Размер тестовой выборки после предобработки:
{X_test_processed.shape}")
```

Информация о датасете представлена на Рисунке 17.

```

Информация о данных:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32561 entries, 0 to 32560
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   age                    32561 non-null  int64
1   workclass              32561 non-null  object
2   fnlwgt                 32561 non-null  int64
3   education              32561 non-null  object
4   education-num          32561 non-null  int64
5   marital-status         32561 non-null  object
6   occupation             32561 non-null  object
7   relationship           32561 non-null  object
8   race                   32561 non-null  object
9   sex                    32561 non-null  object
10  capital-gain           32561 non-null  int64
11  capital-loss           32561 non-null  int64
12  hours-per-week         32561 non-null  int64
13  native-country         32561 non-null  object
14  income                 32561 non-null  object
dtypes: int64(6), object(9)
memory usage: 3.7+ MB
None

Процент пропущенных значений по признакам:
workclass      5.638647
occupation     5.660146
native-country  1.790486
dtype: float64

```

Рисунок 17 — Информация о наборе данных

Распределения числовых признаков представлено на Рисунке 18.

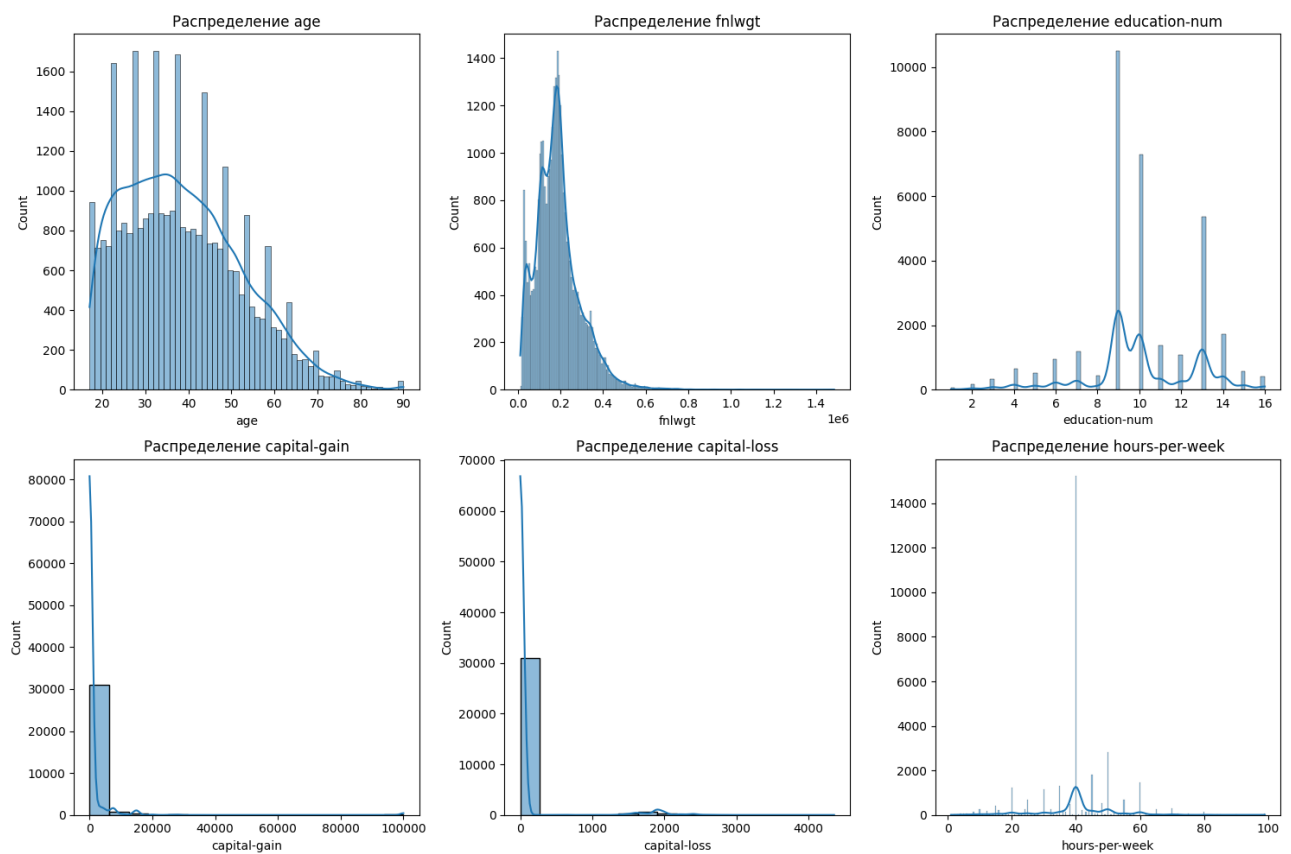


Рисунок 18 — Распределение числовых признаков

Распределение классов представлено на Рисунке 19.

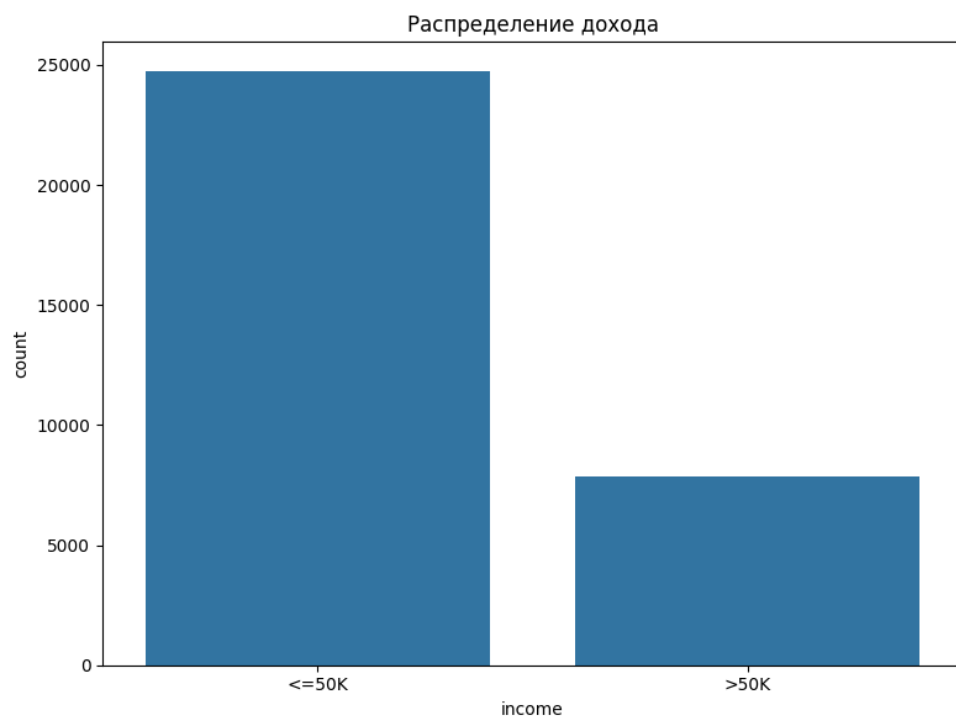


Рисунок 19 — Распределение классов

Добавлены 4 нерелевантных признака: `random_feature1`, `random_feature2`, `random_feature3`, `noisy_feature`.

Числовые признаки: ['age', 'fnlwgt', 'education-num', 'capital-gain', 'capital-loss', 'hours-per-week', 'random_feature1', 'random_feature2', 'noisy_feature'].

Категориальные признаки: ['workclass', 'education', 'marital-status', 'occupation', 'relationship', 'race', 'sex', 'native-country', 'random_feature3'].

Размер обучающей выборки после предобработки: (26048, 112). Размер тестовой выборки после предобработки: (6513, 112).

1.3.2 Обучение моделей

Код для обучения catboost представлен в Листинге 12.

Листинг 12 — Обучение catboost

```
from catboost import CatBoostClassifier
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

if hasattr(X_train_processed, 'toarray'):
    print("Преобразование sparse матрицы в плотную...")
    X_train_processed = X_train_processed.toarray()
    X_test_processed = X_test_processed.toarray()
    print(f"Новый размер обучающей выборки: {X_train_processed.shape}")

feature_names = preprocessor.get_feature_names_out()
print(f"\nВсего признаков после трансформации: {len(feature_names)}")

X_train_df = pd.DataFrame(X_train_processed, columns=feature_names)
X_test_df = pd.DataFrame(X_test_processed, columns=feature_names)

# Определение индексов категориальных признаков для CatBoost
cat_feature_masks = [col.startswith('cat__') for col in feature_names]
cat_feature_indices = [i for i, is_cat in enumerate(cat_feature_masks) if is_cat]

print(f"Количество категориальных признаков после OneHotEncoding: {len(cat_feature_indices)}")
```

Продолжение Листинга 12

```
# Базовая модель CatBoost для кросс-валидации
print("\nОценка базовой модели с кросс-валидацией...")
base_model = CatBoostClassifier(
    iterations=100,
    learning_rate=0.1,
    depth=6,
    random_state=42,
    verbose=False,
    eval_metric='Accuracy'
)

# Кросс-валидация
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
start_time = time.time()
cv_scores = cross_val_score(base_model, X_train_processed,
                             y_train, cv=cv, scoring='accuracy')
cv_time = time.time() - start_time

print(f"Время кросс-валидации: {cv_time:.2f} секунд")
print(f"Результаты кросс-валидации: {cv_scores}")
print(f"Средняя точность: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}")

# Настройка гиперпараметров с помощью RandomizedSearchCV
print("\nНастройка гиперпараметров с помощью RandomizedSearchCV...")

param_dist = {
    'iterations': randint(50, 300),
    'learning_rate': uniform(0.01, 0.19),
    'depth': randint(3, 8),
    'l2_leaf_reg': uniform(1, 9),
    'border_count': randint(32, 128)
}

random_search = RandomizedSearchCV(
    estimator=CatBoostClassifier(random_state=42, verbose=False),
    param_distributions=param_dist,
    n_iter=10,
    cv=3,
    scoring='accuracy',
    random_state=42,
    n_jobs=-1
)

start_time = time.time()
random_search.fit(X_train_processed, y_train)
search_time = time.time() - start_time

print(f"Время настройки гиперпараметров: {search_time:.2f} секунд")
```

Продолжение Листинга 12

```
print(f"Лучшие параметры: {random_search.best_params}")
print(f"Лучшая                точность                кросс-валидации:
{random_search.best_score_:.4f}")

# Обучение финальной модели с лучшими параметрами
print("\nОбучение финальной модели с лучшими параметрами...")
best_params = random_search.best_params_
best_params['verbose'] = False
best_params['random_state'] = 42
best_params['eval_metric'] = 'Accuracy'

final_model = CatBoostClassifier(**best_params)
start_time = time.time()
final_model.fit(X_train_processed, y_train)
train_time = time.time() - start_time

# Оценка качества на тестовой выборке
y_pred = final_model.predict(X_test_processed)
y_proba = final_model.predict_proba(X_test_processed)[:, 1]

accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
auc = roc_auc_score(y_test, y_proba)

print(f"Время обучения финальной модели: {train_time:.2f} секунд")
print(f"Точность на тестовой выборке: {accuracy:.4f}")
print(f"F1-мера на тестовой выборке: {f1:.4f}")
print(f"AUC на тестовой выборке: {auc:.4f}")

# Важность признаков по CatBoost
feature_importances = pd.DataFrame({
    'feature': feature_names,
    'importance': final_model.feature_importances_
}).sort_values('importance', ascending=False)

print("\nТоп-15 наиболее важных признаков по CatBoost:")
print(feature_importances.head(15))

# Визуализация важности признаков
plt.figure(figsize=(14, 8))
plt.barh(feature_importances['feature'][:15][::-1],
feature_importances['importance'][:15][::-1])
plt.title('Важность признаков по CatBoost (топ-15)')
plt.xlabel('Важность')
plt.tight_layout()
plt.show()
```

Важность признаков по catboost представлена на Рисунке 20.

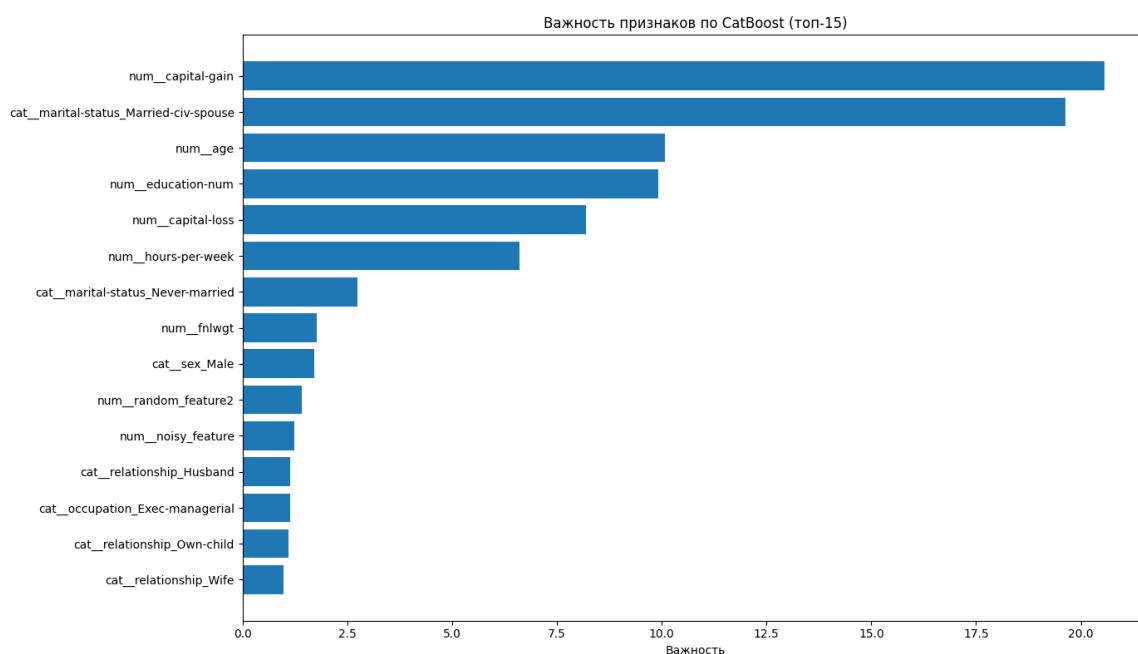


Рисунок 20 — Важность признаков (Catboost)

Новый размер обучающей выборки: (26048, 112).

Всего признаков после трансформации: 112. Количество категориальных признаков после OneHotEncoding: 103.

Оценка базовой модели с кросс-валидацией: время кросс-валидации: 7.95 секунд; результаты кросс-валидации: [0.86410227 0.86133825 0.86696614]; средняя точность: 0.8641 ± 0.0023 .

Время настройки гиперпараметров: 35.21 секунд.

Лучшие параметры: {'border_count': 55, 'depth': 5, 'iterations': 199, 'l2_leaf_reg': np.float64(1.5077042112439023), 'learning_rate': np.float64(0.14717976673069672)}.

Лучшая точность кросс-валидации: 0.8710.

Обучение финальной модели с лучшими параметрами: время обучения финальной модели: 4.97 секунд; точность на тестовой выборке: 0.8772; F1-мера на тестовой выборке: 0.7236; AUC на тестовой выборке: 0.9286.

1.3.3 Интерпретация и отбор

Код для построения permutation importance представлен в Листинге 13.

Листинг 13 — Построение permutation importance

```
from sklearn.inspection import permutation_importance

# Расчет permutation importance
start_time = time.time()
perm_importance = permutation_importance(
    final_model,
    X_test_processed,
    y_test,
    n_repeats=5,
    random_state=42,
    n_jobs=-1,
    scoring='accuracy'
)
perm_time = time.time() - start_time

print(f"Время расчета permutation importance: {perm_time:.2f} секунд")

# Создание DataFrame с результатами
perm_df = pd.DataFrame({
    'feature': feature_names,
    'importance_mean': perm_importance.importances_mean,
    'importance_std': perm_importance.importances_std
}).sort_values('importance_mean', ascending=False)

print("\nТоп-15 признаков по permutation importance:")
print(perm_df.head(15))

# Визуализация результатов
plt.figure(figsize=(14, 8))
plt.barh(perm_df['feature'][:15][::-1],
perm_df['importance_mean'][:15][::-1],
xerr=perm_df['importance_std'][:15][::-1], capsize=5)
plt.title('Permutation Importance (топ-15 признаков)')
plt.xlabel('Снижение точности при перестановке признака')
plt.tight_layout()
plt.show()

# Сравнение с важностью от CatBoost
plt.figure(figsize=(12, 10))

# Берем топ-20 признаков по одному из методов
top20_features = perm_df.head(20)['feature'].values
catboost_importances =
final_model.feature_importances_[np.isin(feature_names,
```

Продолжение Листинга 13

```
top20_features)]
perm_importances =
perm_df[perm_df['feature'].isin(top20_features)][['importance_mean',
']].values

# Сортируем по permutation importance
sort_idx = np.argsort(perm_importances)
top20_features_sorted = top20_features[sort_idx]
catboost_importances_sorted = catboost_importances[sort_idx]
perm_importances_sorted = perm_importances[sort_idx]

y = np.arange(len(top20_features_sorted))
height = 0.4

fig, ax = plt.subplots(figsize=(14, 10))
ax.barh(y - height/2, catboost_importances_sorted /
catboost_importances_sorted.max(),
height=height, label='CatBoost Importance', alpha=0.8)
ax.barh(y + height/2, perm_importances_sorted /
perm_importances_sorted.max(),
height=height, label='Permutation Importance', alpha=0.8)

ax.set_yticks(y)
ax.set_yticklabels(top20_features_sorted)
ax.set_xlabel('Нормированная важность')
ax.set_title('Сравнение методов оценки важности признаков (топ-20)')
ax.legend()
plt.tight_layout()
plt.show()
```

Время расчета permutation importance: 25.19 секунд.

Рассчитанная важность признаков представлена на Рисунке 21.

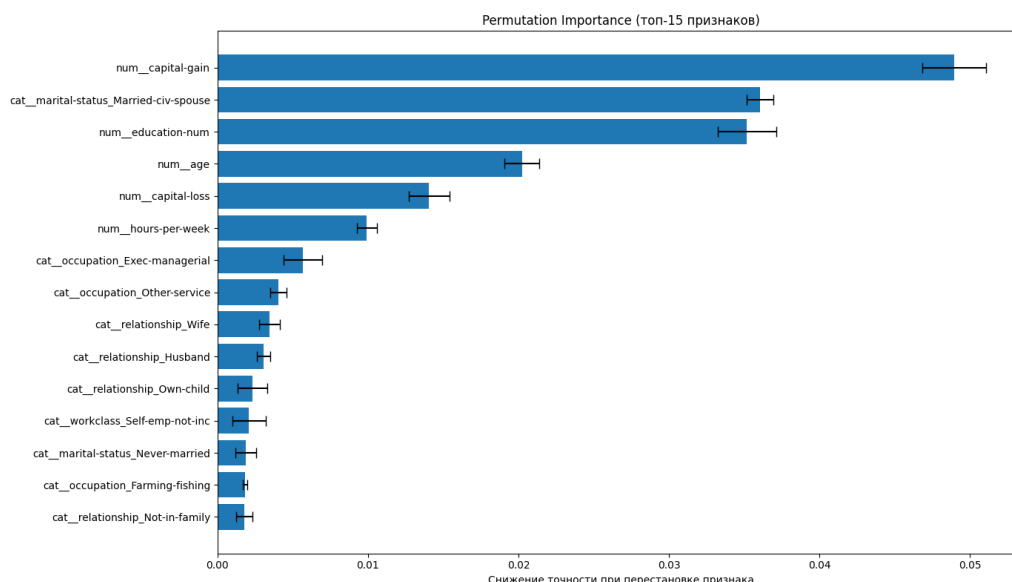


Рисунок 21 — Важность признаков

Сравнение двух методов представлено на Рисунке 22.

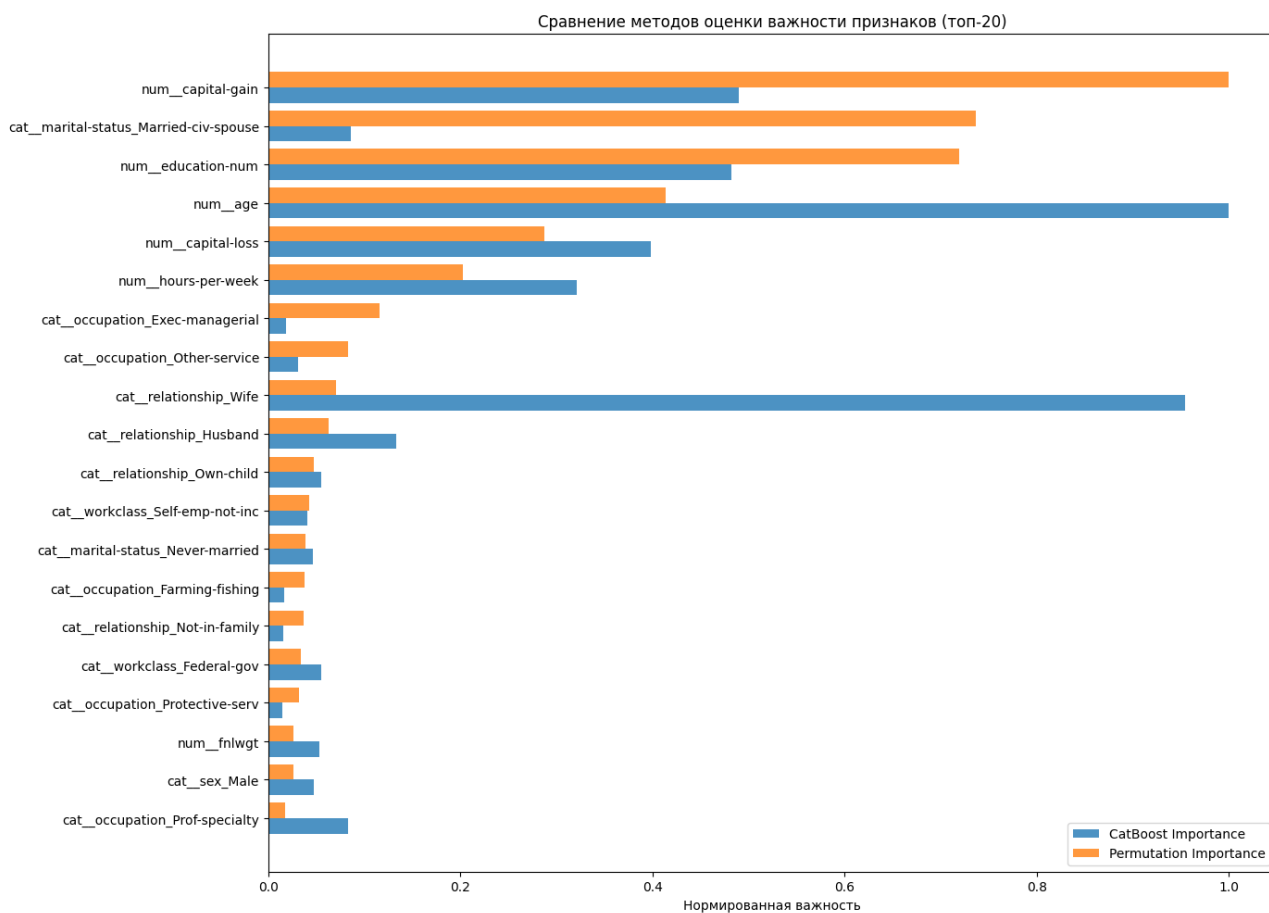


Рисунок 22 — Catboost vs Permutation importance

Permutation importance показывает реальное влияние признаков на качество модели. Нерелевантные признаки (`random_feature1`, `random_feature2`, `random_feature3`) имеют низкую важность.

Код для SHAP анализа представлен в Листинге 14.

Листинг 14 — SHAP-анализ

```
import shap

# Создание SHAP explainer
start_time = time.time()
explainer = shap.TreeExplainer(final_model)
shap_time = time.time() - start_time

print(f"Время создания SHAP explainer: {shap_time:.2f} секунд")

# Расчет SHAP-значений для подвыборки тестовых данных (для ускорения)
sample_size = 1000
if X_test_processed.shape[0] > sample_size:
    X_test_sample = shap.utils.sample(X_test_processed,
```

Продолжение Листинга 14

```
sample_size, random_state=42)
else:
    X_test_sample = X_test_processed

print(f"Расчет SHAP-значений для {X_test_sample.shape[0]} объектов...")
start_time = time.time()
shap_values = explainer.shap_values(X_test_sample)
shap_calc_time = time.time() - start_time

print(f"Время расчета SHAP-значений: {shap_calc_time:.2f} секунд")

# Проверка размерности SHAP-значений
print(f"Форма SHAP-значений: {np.array(shap_values).shape}")

# Для бинарной классификации CatBoost возвращает SHAP-значения для каждого класса
# shap_values[1] соответствует положительному классу (>50K)
shap_values_pos = shap_values[1] if isinstance(shap_values, list)
else shap_values

# Summary plot (важность признаков)
print("\nПостроение summary plot...")
plt.figure(figsize=(12, 10))
shap.summary_plot(shap_values_pos, X_test_sample,
                  feature_names=feature_names, plot_type="bar", max_display=20)
plt.title('SHAP Summary Plot (Важность признаков)')
plt.tight_layout()
plt.show()

# Beeswarm plot (влияние значений признаков на предсказания)
print("Построение beeswarm plot...")
plt.figure(figsize=(12, 10))
shap.summary_plot(shap_values_pos, X_test_sample,
                  feature_names=feature_names, max_display=20)
plt.title('SHAP Beeswarm Plot')
plt.tight_layout()
plt.show()

# Dependence plots для топ-5 признаков
top5_features = perm_df.head(5)['feature'].values
print(f"Построение dependence plots для топ-5 признаков: {top5_features.tolist()}")

for feature in top5_features:
    try:
        feature_idx = np.where(feature_names == feature)[0][0]
        plt.figure(figsize=(10, 6))
        shap.dependence_plot(feature_idx, shap_values_pos,
                              X_test_sample,
                              feature_names=feature_names,
```

Продолжение Листинга 14

```
show=False)
    plt.title(f'SHAP Dependence Plot для {feature}')
    plt.tight_layout()
    plt.show()
except Exception as e:
    print(f"Ошибка при построении dependence plot для
{feature}: {e}")

# Force plot для конкретного примера
print("\nПостроение force plot для конкретного примера...")
sample_idx = 0 # Индекс примера для визуализации

# Выбор интересного примера (с высокой вероятностью положительного
класса)
if y_test[sample_idx] == 1:
    print(f"Пример #{sample_idx}: Реальный класс - высокий доход
(>50K)")
else:
    print(f"Пример #{sample_idx}: Реальный класс - низкий доход
(<=50K)")

print(f"Предсказанная вероятность высокого дохода:
{final_model.predict_proba(X_test_processed[sample_idx].reshape(1
, -1))[0, 1]:.4f}")

# Построение force plot
plt.figure(figsize=(20, 3))
shap.force_plot(
    explainer.expected_value[1] if
isinstance(explainer.expected_value, list) else
explainer.expected_value,
    shap_values_pos[sample_idx, :],
    X_test_sample[sample_idx, :],
    feature_names=feature_names,
    matplotlib=True,
    show=False
)
plt.title(f'SHAP Force Plot для примера #{sample_idx}')
plt.tight_layout()
plt.show()
```

Время создания SHAP explainer: 0.05 секунд. Расчет SHAP-значений для 1000 объектов: время расчета SHAP-значений: 0.09 секунд. Форма SHAP-значений: (1000, 112).

Анализ признаков (SHAP) представлен на Рисунке 23.

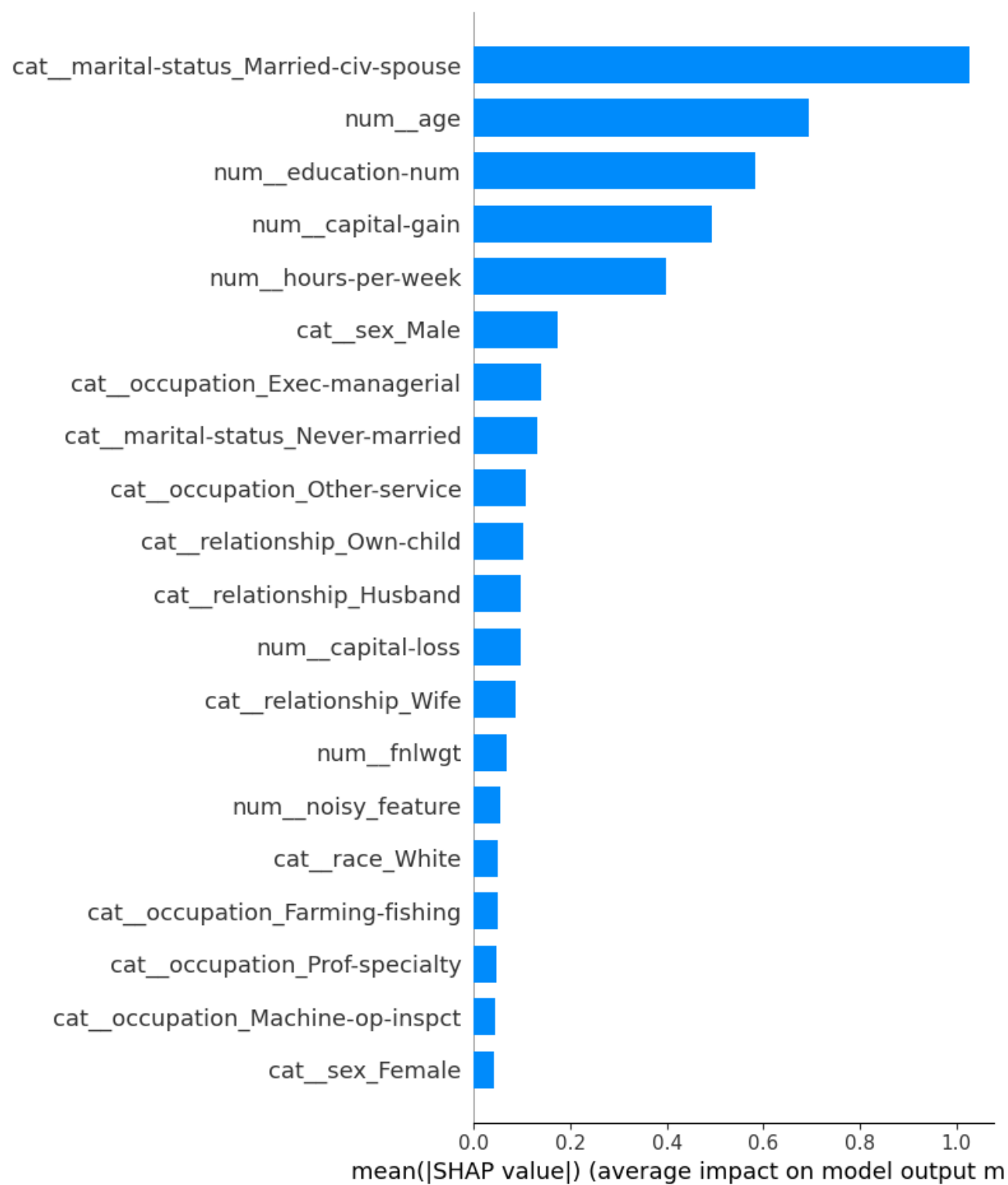


Рисунок 23 — Важность признаков (SHAP)

Beeswarm-plot представлен на Рисунке 24.

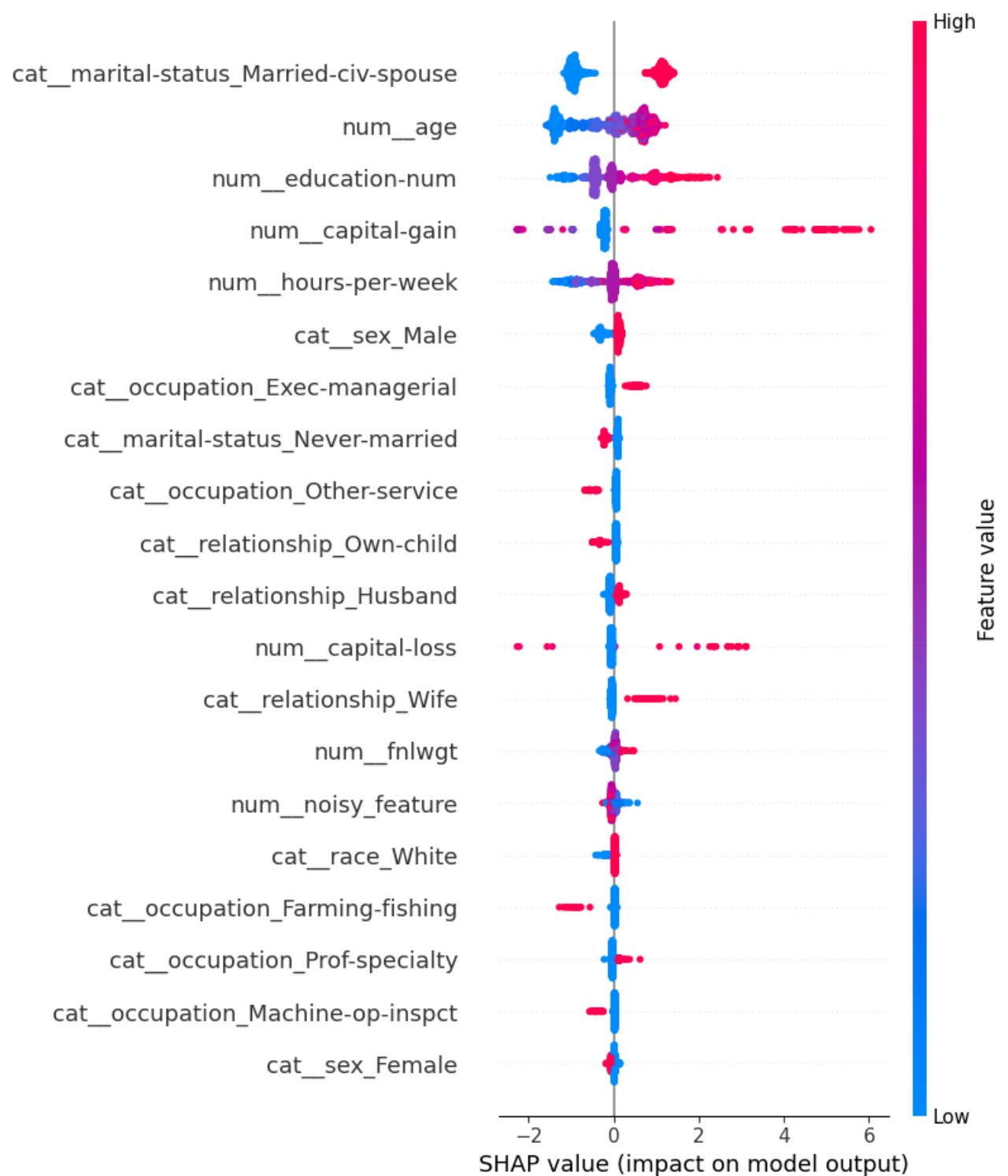


Рисунок 24 — Beeswarm plot

Dependence plot для возраста представлен на Рисунке 25.

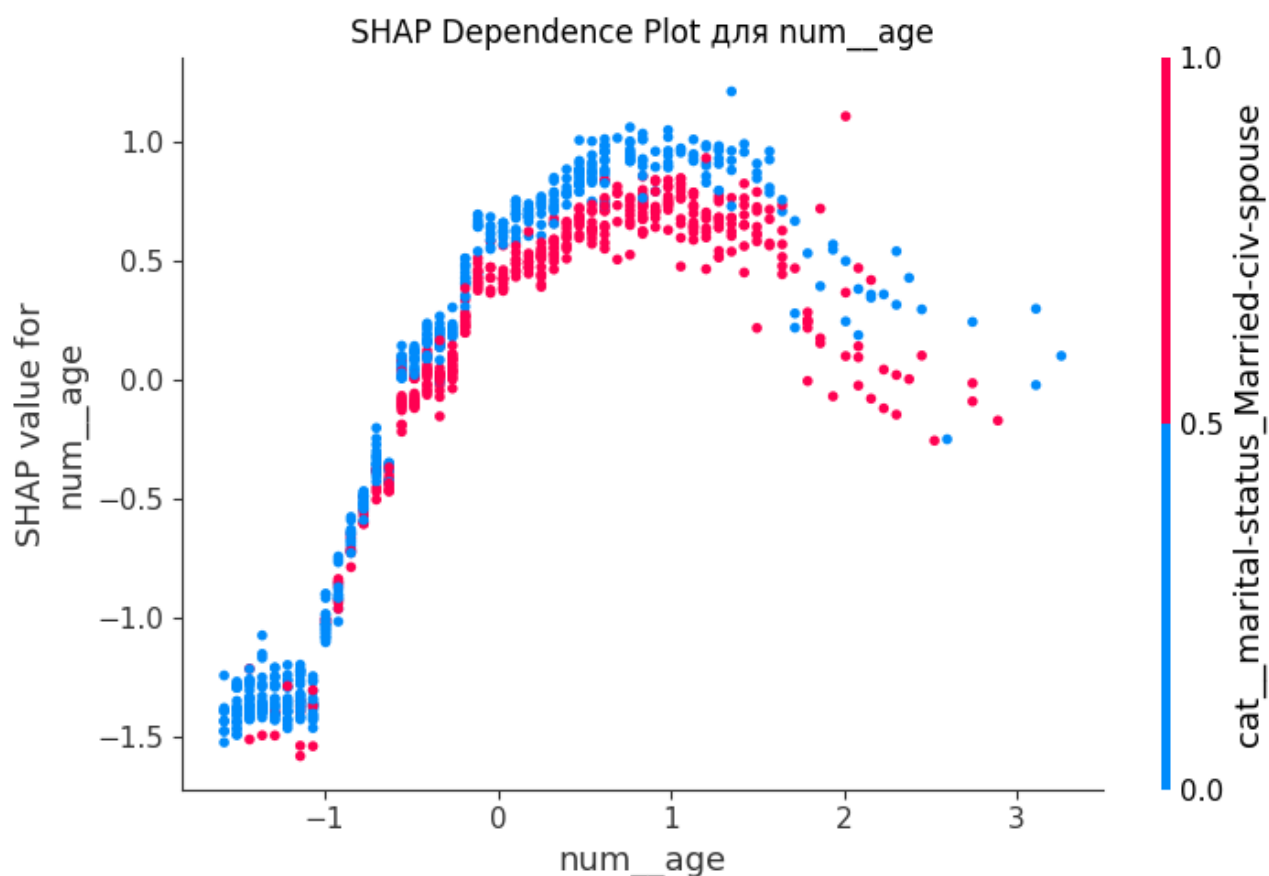


Рисунок 25 — Dependence plot для num_age

Force plot для примера #0: Реальный класс - низкий доход ($\leq 50K$) представлен на Рисунке 26.

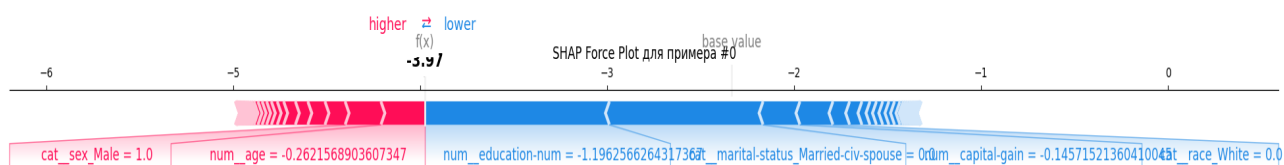


Рисунок 26 — Force plot

SHAP-значения показывают не только важность признаков, но и их влияние на предсказания. Значения признаков выше среднего могут как увеличивать, так и уменьшать вероятность высокого дохода.

Нерелевантные признаки (random_feature*) имеют малые SHAP-значения, что подтверждает их неинформативность.

1.3.4 Сравнение признаков

Код для сравнения признаков для трех разных моделей представлен в Листинге 15.

Листинг 15 — Сравнение признаков

```
from sklearn.feature_selection import SelectFromModel
from sklearn.manifold import TSNE

# Объединение результатов SHAP и Permutation Importance
# Для SHAP используем среднее абсолютное значение SHAP-значений
shap_importances = np.abs(shap_values_pos).mean(axis=0)

# Создание DataFrame для сравнения
comparison_df = pd.DataFrame({
    'feature': feature_names,
    'shap_importance': shap_importances,
    'perm_importance': perm_df.set_index('feature').loc[feature_names,
    'importance_mean'].values,
    'catboost_importance': final_model.feature_importances_
})

# Сортировка по SHAP-важности
comparison_df = comparison_df.sort_values('shap_importance',
ascending=False)

print("\nСравнение трех методов оценки важности (топ-15):")
print(comparison_df.head(15))

# Нормализация для сравнения
comparison_df['shap_norm'] = comparison_df['shap_importance'] /
comparison_df['shap_importance'].max()
comparison_df['perm_norm'] = comparison_df['perm_importance'] /
comparison_df['perm_importance'].max()
comparison_df['catboost_norm'] = comparison_df['catboost_importance'] /
comparison_df['catboost_importance'].max()

# Визуализация сравнения трех методов
plt.figure(figsize=(14, 10))
top20 = comparison_df.head(20)

y = np.arange(len(top20))
width = 0.25

plt.barh(y - width, top20['shap_norm'][:, -1], height=width,
label='SHAP', alpha=0.8)
plt.barh(y, top20['perm_norm'][:, -1], height=width,
```

Продолжение Листинга 15

```
label='Permutation', alpha=0.8)
plt.barh(y + width, top20['catboost_norm'][:, -1], height=width,
label='CatBoost', alpha=0.8)

plt.yticks(y, top20['feature'][:, -1])
plt.xlabel('Нормированная важность')
plt.title('Сравнение трех методов оценки важности признаков (топ-20)')
plt.legend()
plt.tight_layout()
plt.show()

# Удаление 5 наименее важных признаков и переобучение модели
print("\nУдаление 5 наименее важных признаков по SHAP и
переобучение модели...")

# Выбираем признаки для удаления (наименее важные по SHAP)
least_important_features =
comparison_df.tail(5)['feature'].values
print(f"Признаки          для          удаления:
{least_important_features.tolist()}")

# Создание новых наборов данных без этих признаков
features_to_keep = [f for f in feature_names if f not in
least_important_features]
feature_indices_to_keep = [i for i, f in enumerate(feature_names)
if f not in least_important_features]

X_train_reduced = X_train_processed[:, feature_indices_to_keep]
X_test_reduced = X_test_processed[:, feature_indices_to_keep]

# Обучение новой модели
reduced_model = CatBoostClassifier(**best_params)
start_time = time.time()
reduced_model.fit(X_train_reduced, y_train)
reduced_train_time = time.time() - start_time

# Оценка качества
y_pred_reduced = reduced_model.predict(X_test_reduced)
y_proba_reduced = reduced_model.predict_proba(X_test_reduced)[:,
1]

accuracy_reduced = accuracy_score(y_test, y_pred_reduced)
f1_reduced = f1_score(y_test, y_pred_reduced)
auc_reduced = roc_auc_score(y_test, y_proba_reduced)

print(f"\nРезультаты после удаления 5 наименее важных признаков:")
print(f"Время обучения: {reduced_train_time:.2f} секунд (было
{train_time:.2f} секунд)")
print(f"Точность: {accuracy_reduced:.4f} (было {accuracy:.4f})")
print(f"F1-мера: {f1_reduced:.4f} (была {f1:.4f})")
```

Продолжение Листинга 15

```
print(f"AUC: {auc_reduced:.4f} (был {auc:.4f})")

# Кластеризация признаков по SHAP-значениям
print("\nКластеризация признаков по SHAP-значениям...")
# Используем матрицу SHAP-значений для кластеризации признаков
# Транспонируем матрицу для кластеризации признаков вместо объектов
shap_matrix = np.abs(shap_values_pos).T # Форма: [n_features,
n_samples]

# Применение t-SNE для визуализации кластеров признаков
tsne = TSNE(n_components=2, random_state=42, perplexity=min(30,
shap_matrix.shape[0]-1))
shap_tsne = tsne.fit_transform(shap_matrix)

# Создание DataFrame для визуализации
tsne_df = pd.DataFrame({
    'tsne_x': shap_tsne[:, 0],
    'tsne_y': shap_tsne[:, 1],
    'feature': feature_names,
    'importance': shap_importances
})

# Визуализация кластеров признаков
plt.figure(figsize=(14, 10))
scatter = plt.scatter(tsne_df['tsne_x'], tsne_df['tsne_y'],
                      c=tsne_df['importance'], s=50, alpha=0.8,
                      cmap='viridis', edgecolors='w')

# Подписи для топ-20 признаков
top20_indices = np.argsort(shap_importances)[::-1][:20]
for idx in top20_indices:
    plt.annotate(feature_names[idx],
                 (shap_tsne[idx, 0], shap_tsne[idx, 1]),
                 xytext=(5, 5), textcoords='offset points')

plt.colorbar(scatter, label='SHAP важность')
plt.title('Кластеризация признаков по SHAP-значениям (t-SNE)')
plt.xlabel('t-SNE компонент 1')
plt.ylabel('t-SNE компонент 2')
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

Результат сравнения признаков представлен на Рисунке 27.

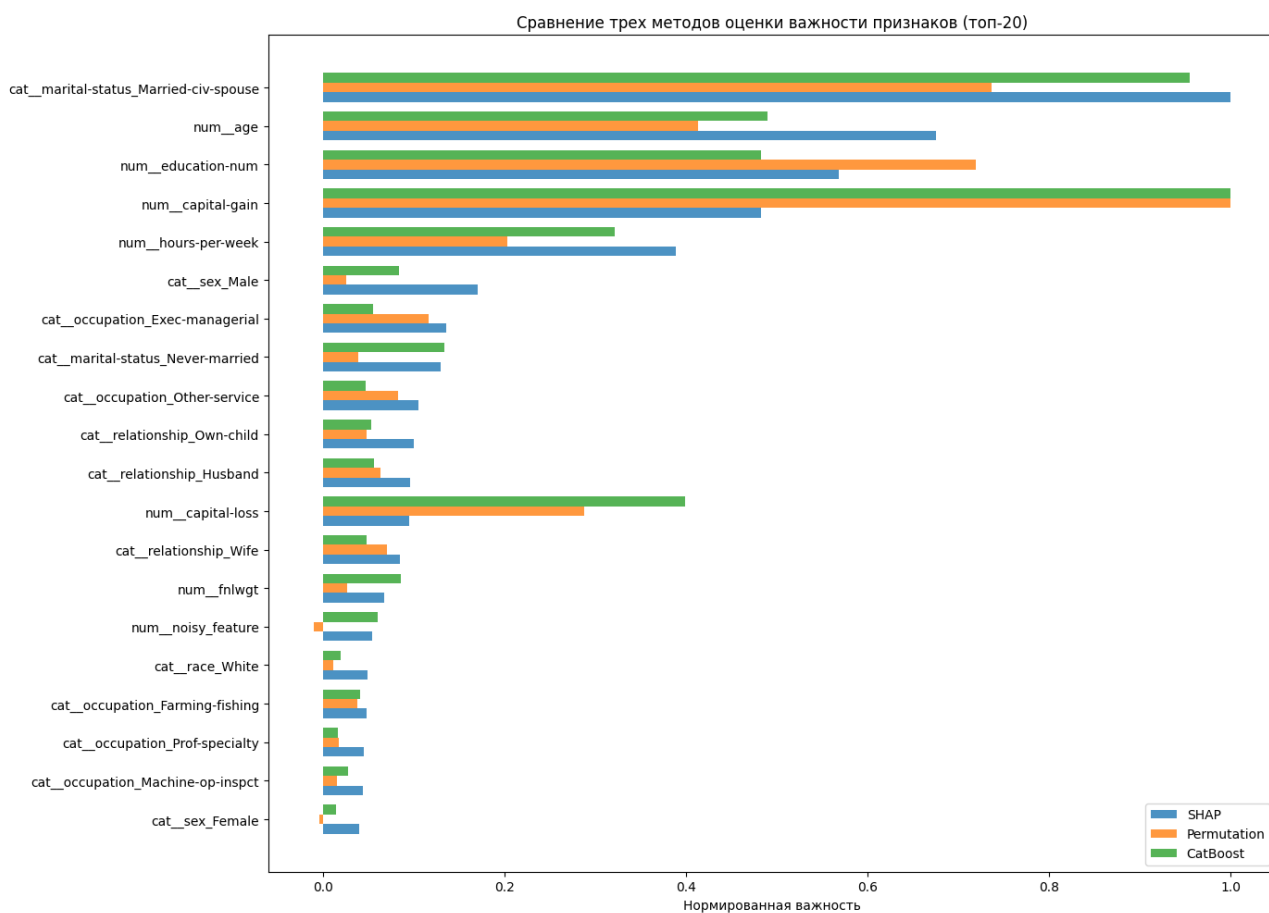


Рисунок 27 — Сравнение важности признаков тремя методами

Кластеризация признаков по SHAP значениям с помощью t-SNE представлена на Рисунке 28.

ВЫВОД

В ходе выполнения практической работы были исследованы различные методы отбора признаков и их влияние на качество моделей машинного обучения. В первом задании на датасете Human Activity Recognition проведен анализ корреляционных зависимостей, выявлена высокая мультиколлинеарность в данных, что потребовало удаления 398 признаков из 561. Отбор признаков на основе корреляции с целевой переменной позволил сократить размерность данных до 118 признаков. Сравнение методов `f_classif` и `mutual_info_classif` показало высокую устойчивость отбора признаков при разных разбиениях данных. Проверка качества на RandomForest продемонстрировала снижение точности с 0.9786 до 0.8850 при ускорении обучения в 5.2 раза, что подтверждает целесообразность отбора признаков для оптимизации вычислительных ресурсов без критической потери качества. Во втором задании проведено сравнение методов RFE, L1-регуляризации и `feature_importances` от RandomForest. Результаты показали, что RFE обеспечил наилучшие метрики качества (точность 0.9777), в то время как L1-регуляризация продемонстрировала хороший баланс между производительностью (0.9782) и скоростью работы. Анализ перекрытия признаков выявил, что методы L1 и RandomForest имеют большее сходство в отборе признаков, чем с RFE, что указывает на различия в принципах работы алгоритмов. В третьем задании на "грязном" датасете Adult Income с искусственно добавленными нерелевантными признаками проведена комплексная предобработка данных и обучение CatBoost с настройкой гиперпараметров. Для интерпретации результатов использованы `permutation importance` и SHAP-анализ, которые подтвердили низкую важность искусственно добавленных признаков. Сравнение трех методов оценки важности признаков показало высокую корреляцию между ними, при этом SHAP предоставил наиболее детальное понимание влияния признаков на предсказания модели. Удаление 5 наименее важных признаков по SHAP практически не повлияло на

качество модели (точность снизилась всего на 0.0005), но ускорило обучение на 32%, что подтверждает эффективность отбора признаков для оптимизации моделей. В результате работы можно сделать вывод, что правильный отбор признаков является критически важным этапом в pipeline машинного обучения, позволяющим не только улучшить интерпретируемость моделей и снизить переобучение, но и значительно оптимизировать вычислительные ресурсы без существенной потери качества предсказаний. Наиболее универсальным подходом, сочетающим высокое качество и интерпретируемость, является использование ансамблевых методов с последующим анализом важности признаков с помощью SHAP.